

CSE357: COMBINATORIAL STUDIES

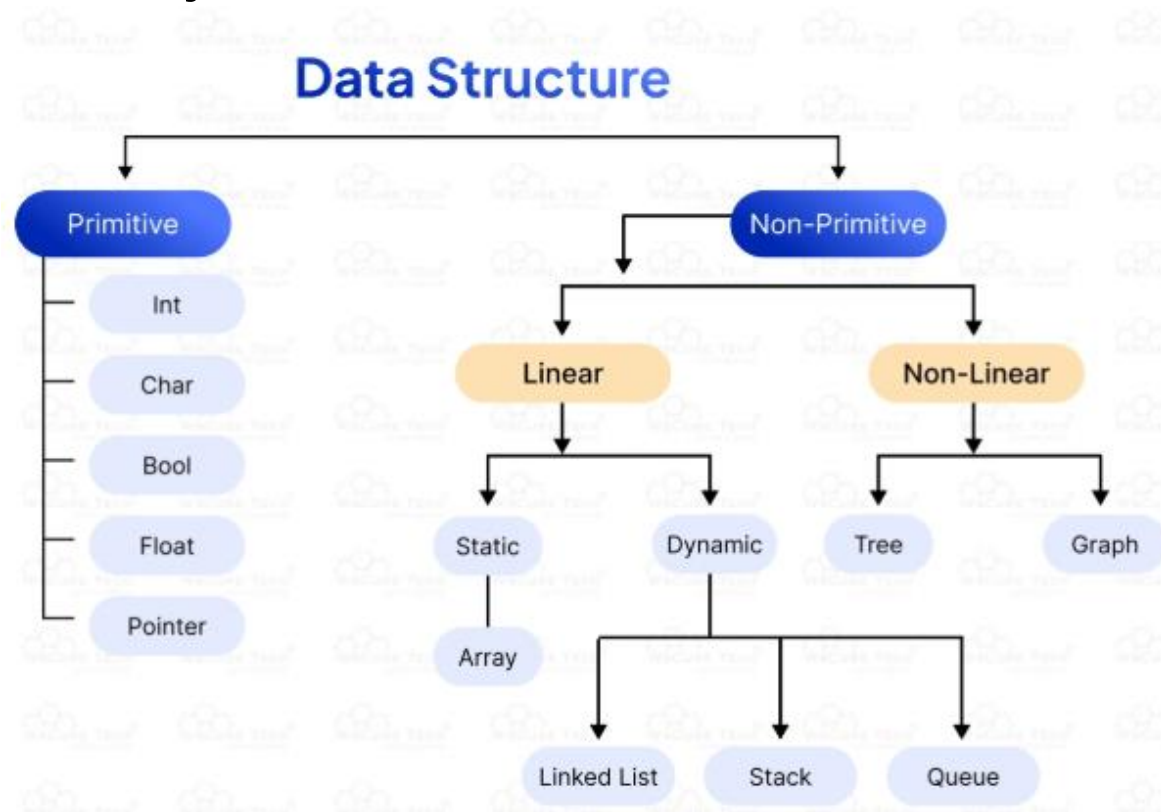
Dr. Harwinder Singh Sohal
Computer Science & Engg.

Unit V: Algorithms and Data Structures

- Algorithms and Data Structures
- Understanding Algorithms and Analysis
- Running Time Analysis and Rate of Growth
- Asymptotic Notation: Big-O Notation
- Data Structures and Recursive Algorithms
- Recursion and Backtracking,, Linked Lists
- Stacks and Queues
- Trees and Binary Search Trees (BSTs)

DATA STRUCTURE

A data structure is a way of organizing and storing data in memory so that it can be accessed and modified efficiently.



Primitive Data Structures

These are **basic data types** that store **simple values**.

☞ Examples:

- Integer (int)
- Float (float)
- Character (char)
- Boolean (bool)

☞ Features:

- Store **single value**
- Simple and basic
- Directly supported by programming languages

Non-Primitive Data Structures

These are **complex data structures** that store **multiple values** and organize them efficiently.

☞ **Types:**

(a) Linear Data Structures

- Array
- Stack
- Queue
- Linked List

(b) Non-Linear Data Structures

- Tree
- Graph

An **array** is a **collection of elements** stored in **contiguous memory locations**, and each element can be accessed using an **index (position number)**.

Index	Value (Marks)
0	78
1	85
2	90
3	66
4	72



Key Features of an Array:

- Stores multiple values
 - All values are of the same data type
 - Uses indexing to access elements
 - Fast access time
-

Real-Life Example:

- Days of the week → Monday, Tuesday, Wednesday, ...
- Roll numbers of students
- Temperatures of a week

Advantages:

- Easy to manage multiple values
 - Faster access using index
 - Saves memory compared to using many separate variables
-

Disadvantages:

- Fixed size (in most languages)
- Insertion and deletion can be difficult
- Only same type of data can be stored

Simple Definition:

A **1D array** stores multiple values of the same data type in a single line, using **one index** to access each value.

Easy Example (Without Code):

Suppose we store marks of 5 students:

Index	0	1	2	3	4
Value	75	82	90	68	88

This is a **1D array** because:

- Data is stored in **one line**
- Only **one index** is used

Real-Life Examples of 1D Array:

- Days in a week → [Mon, Tue, Wed, Thu, Fri, Sat, Sun]
 - Roll numbers of students
 - Monthly temperatures
 - Phone contact list
-

★ Key Features of 1D Array:

- Linear structure
- Uses single index
- Fast access
- Easy to understand

A 2D array stores multiple values of the same data type in the form of rows and columns, and each element is accessed using two indexes — one for row and one for column.

Easy Example (Without Code):

Suppose we store marks of 3 students in 3 subjects:

Student / Subject	Math	Science	English
Student 1	78	85	80
Student 2	90	88	92
Student 3	65	70	68

Real-Life Examples of 2D Array:

- Class marks sheet
 - Chess board (8×8 grid)
 - Calendar (weeks $\times$ days)
 - Cinema seat arrangement
 - Image pixels (rows $\times$ columns)
-

★ Key Features of 2D Array:

- Data stored in table format
- Uses two indexes
- Easy to represent matrices and grids

Time Complexity

Operation	Time Complexity
Access	$O(1)$
Traversal	$O(n)$
Search (Linear)	$O(n)$
Search (Binary)	$O(\log n)$
Insertion (End)	$O(1)$
Insertion (Middle/Start)	$O(n)$
Deletion (End)	$O(1)$
Deletion (Middle/Start)	$O(n)$

In an array of size n , the time complexity of accessing the i -th element is:

$$O(1)$$

Reason:

- Arrays store elements in **contiguous memory locations**.
- Using the **index (i)**, the memory address of the element can be calculated **directly**.
- So, **no looping or searching is needed**.

Key Point:

Accessing any element in an array — **first, last, or i-th** — always takes **constant time**.

Row-major order is a way of storing elements of a 2D array in memory, where **all elements of one row are stored first, then the next row, and so on.**

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Easy Example:

Consider a 2D array:

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Row-wise reading:

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9

 This is row-major order.

How Data is Stored in Memory:

Memory order will be:

1, 2, 3, 4, 5, 6, 7, 8, 9

Address Calculation Formula (Concept):

For a 2D array $A[m][n]$,

Address of $A[i][j]$ in row-major order:

$$\text{Address} = \text{Base Address} + ((i \times n + j) \times \text{Size of each element})$$

Where:

- i = row index
- j = column index
- n = total number of columns

Column-major order is a way of storing elements of a 2D array in memory, where **all elements of one column** are stored first, then the next column, and so on.

s	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

How Data is Stored in Memory:

Memory order will be:

1, 4, 7, 2, 5, 8, 3, 6, 9

Address Calculation Formula (Concept):

For a 2D array $A[m][n]$,

Address of $A[i][j]$ in column-major order:

$$\text{Address} = \text{Base Address} + ((j \times m + i) \times \text{Size of each element})$$

Where:

- i = row index
- j = column index
- m = total number of rows

 Given:

- Array: $A[4][5]$
- Base Address (BA) = 2000
- Size of each element (W) = 4 bytes
- Element to find: $A[2][3]$
- Storage order: Row-major

 Row-Major Address Formula:

For $A[m][n]$, the address of $A[i][j]$ is:

$$\text{Address} = \text{BA} + ((i \times n + j) \times W)$$

Where:

- i = row index = 2
- j = column index = 3
- n = number of columns = 5
- W = size of each element = 4 bytes

 **Substitute the values:**

$$\begin{aligned}\text{Address} &= 2000 + ((2 \times 5 + 3) \times 4) \\ &= 2000 + ((10 + 3) \times 4) \\ &= 2000 + (13 \times 4) \\ &= 2000 + 52 \\ &= \boxed{2052}\end{aligned}$$

 **Final Answer:**

2052

For a 3D array $A[x][y][z]$ stored in row-major order, memory is stored in this sequence:

👉 First dimension (x) → then y → then z

So, the address calculation formula is:

$$\text{Address} = B + W \times (i \cdot y \cdot z + j \cdot z + k)$$

Where:

- B = Base address
- W = Size of each element
- i = index in x dimension
- j = index in y dimension
- k = index in z dimension
- y, z = size of second and third dimensions

Operations on Array

An array supports several common operations that allow us to store, access, modify, and manage data.

◆ Main Operations on Array

1 Traversal

- Visiting each element one by one.
- Used for printing or processing elements.

Example: Display all elements.

Time Complexity: $O(n)$

2 Insertion

- Adding a **new element** into the array.

Types:

- At beginning → $O(n)$
 - At middle → $O(n)$
 - At end → $O(1)$ (if space available)
-

3 Deletion

- Removing an **existing element**.

Types:

- From beginning → $O(n)$
- From middle → $O(n)$
- From end → $O(1)$

4 Searching

- Finding the position of a given element.

Methods:

- Linear Search $\rightarrow O(n)$
 - Binary Search (sorted array) $\rightarrow O(\log n)$
-

5 Updation (Modification)

- Changing the value of an element.

Example: Replace element at index 2.

Time Complexity: $O(1)$

6 Sorting

- Arranging elements in **ascending or descending order**.

Examples: Bubble sort, Selection sort, Merge sort, Quick sort

Time Complexity: Depends on algorithm (e.g., $O(n^2)$, $O(n \log n)$)

7 Merging

- Combining **two arrays into one**.

Time Complexity: $O(n + m)$

Operation	Description	Time Complexity
Traversal	Visit all elements	$O(n)$
Insertion	Add element	$O(n) / O(1)$
Deletion	Remove element	$O(n) / O(1)$
Searching	Find element	$O(n), O(\log n)$
Updation	Modify element	$O(1)$
Sorting	Arrange elements	$O(n \log n)$
Merging	Combine arrays	$O(n + m)$

Linear search

Simple Definition:

Linear search is a method of searching an element by comparing it sequentially with every element until the match is found or the list ends.

How Linear Search Works:

1. Start from the first element
2. Compare with the key value
3. If match → Stop (Found)
4. Else → Move to next element
5. Continue until found or list ends

Example (Without Code):

Array: [12, 45, 23, 67, 89, 34]

Search for 67

Steps:

- Compare 12 → No
- Compare 45 → No
- Compare 23 → No
- Compare 67 → Yes (Found)

 Position = 4th element

Time Complexity:

Case	Time
Best Case	$O(1)$ (element at first position)
Average Case	$O(n)$
Worst Case	$O(n)$ (element at last or not present)

Space Complexity:

$O(1)$ (no extra memory used)

✓ Advantages:

- Very simple
 - Works on **unsorted data**
 - Easy to implement
-

✗ Disadvantages:

- Slow for large datasets
 - Inefficient compared to binary search
-

◆ Applications:

- Small data sets
- Unsorted lists
- Simple searching tasks

Binary search is a method of finding an element by repeatedly dividing the array into halves until the element is found.

Important Condition:

 The array must be SORTED.

How Binary Search Works:

1. Find the middle element
2. Compare it with the key value
3. If equal → Found
4. If key < middle → search in left half
5. If key > middle → search in right half
6. Repeat until found or range becomes empty

 Time Complexity:

Case	Time
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

 Space Complexity:

- Iterative: $O(1)$
- Recursive: $O(\log n)$

✓ Advantages:

- Very fast
 - Efficient for large datasets
-

✗ Disadvantages:

- Requires sorted data
 - Not suitable for linked lists
-

◆ Applications:

- Searching in databases
- Phone directories
- Dictionary word search
- Searching in large sorted files

Sorted Array:

[10, 20, 30, 40, 50, 60, 70]

Element to Search: 50

Step-by-Step Working:

Step 1:

- Start = 0
- End = 6
- Middle = $(0 + 6) / 2 = 3$

Element at index 3 = 40

Compare:

$50 > 40 \rightarrow$ Search in right half

Step 2:

- Start = 4
- End = 6
- Middle = $(4 + 6) / 2 = 5$

Element at index 5 = 60

Compare:

$50 < 60 \rightarrow$ Search in left half

Step 3:

- Start = 4
- End = 4
- Middle = $(4 + 4) / 2 = 4$

Element at index 4 = 50

 **Element Found**

A **stack** is a **linear data structure** that follows the principle of:

LIFO – Last In, First Out

This means **the element inserted last is removed first**.

Simple Definition:

A **stack** is a data structure in which **insertion and deletion happen only at one end**, called the **TOP**.

Real-Life Example:

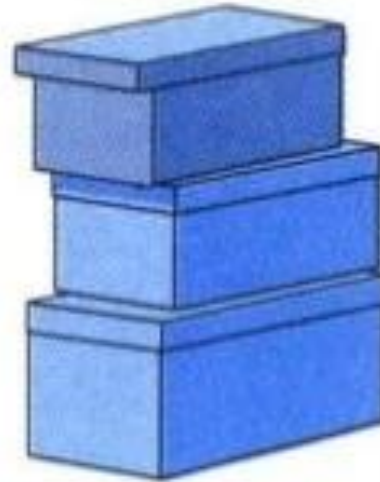
- Stack of **plates**
- Stack of **books**
- Browser **Back button**
- Undo operation in **MS Word**

Some real life examples of Stack

A stack of
cafeteria trays



A stack of
shoe boxes



A stack
of pennies



A stack of
neatly folded shirts

Operation	Meaning
Push	Insert element
Pop	Remove element
Peek / Top	View top element
isEmpty	Check if stack is empty
isFull	Check if stack is full

Time Complexity of Stack Operations:

Operation	Time
Push	$O(1)$
Pop	$O(1)$
Peek	$O(1)$

Stack Implementation:

- Using Array
 - Using Linked List
-

Applications of Stack:

- Expression evaluation (infix $\rightarrow$ postfix)
 - Function calls (call stack)
 - Undo / Redo operations
 - Syntax parsing
 - Reversing a string
-
- In stack (linked list implementation), **top of stack = head of linked list.**
 - To make **push and pop operations $O(1)$** , insertion is done at the **beginning (head)**.

- The **stack data structure** is used to:
 - convert an arithmetic expression from infix to postfix
 - Store **function parameters**
 - Store **local variables**
 - Store **return addresses**

Hence, **recursion is internally managed using a stack.**

- The **maximum depth of a stack frame** depends on the total memory available and the size of each frame.
- If a system has **M bytes** of stack memory and each frame occupies **F bytes**, the maximum number of frames is **M / F**
- Exceeding this limit can cause a **stack overflow**.

To implement a **queue** using **only stacks**, you need **two stacks**:

- 1.Stack 1: Used for enqueue (insertion)**
- 2.Stack 2: Used for dequeue (removal)**

A web browser uses **two stacks** to manage navigation:

1.Back Stack – stores previously visited pages

2.Forward Stack – stores pages that can be revisited using
the forward buttons

$$(a + b) * (c + d)$$

1. Prefix Notation (Polish Notation)

In prefix notation, the operator comes before the operands.

Prefix:

$$* + a b + c d$$

2. Postfix Notation (Reverse Polish Notation)

In postfix notation, the operator comes after the operands.

Postfix:

$$a b + c d + *$$

2 3 * 5 4 * + 9 -

Step	Symbol	Action	Stack
1	2	Push	[2]
2	3	Push	[2, 3]
3	*	Pop 2, 3 $\rightarrow 2 \times 3 = 6 \rightarrow$ Push	[6]
4	5	Push	[6, 5]
5	4	Push	[6, 5, 4]
6	*	Pop 5, 4 $\rightarrow 5 \times 4 = 20 \rightarrow$ Push	[6, 20]
7	+	Pop 6, 20 $\rightarrow 6 + 20 = 26 \rightarrow$ Push	[26]
8	9	Push	[26, 9]
9	-	Pop 26, 9 $\rightarrow 26 - 9 = 17 \rightarrow$ Push	[17]



5 6 + 2 *

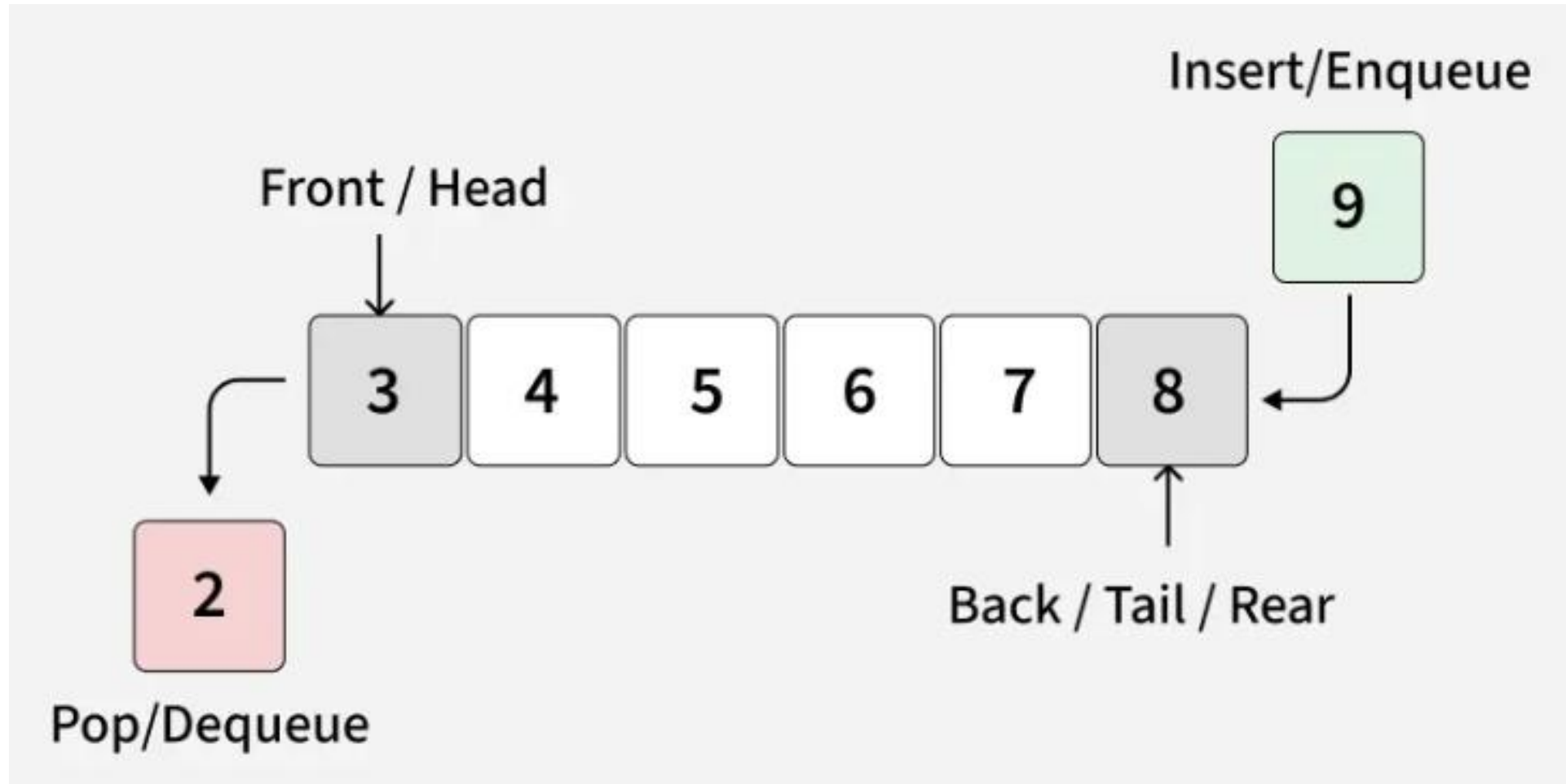
4 5 6 * + 7 -

Queue

A **Queue Data Structure** is a fundamental concept in computer science used for storing and managing data in a specific order.

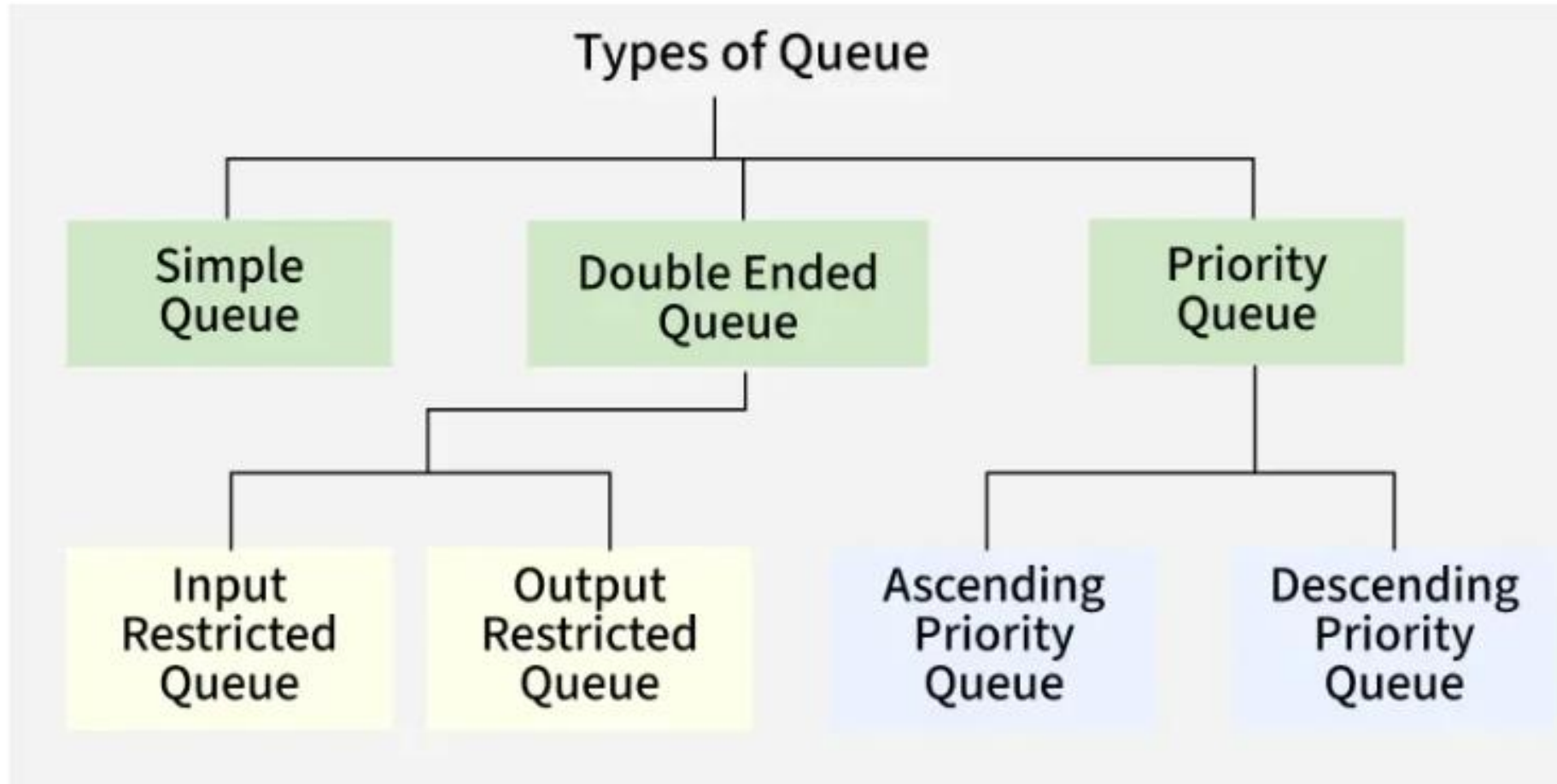
- It follows the principle of "**First in, First out**" (**FIFO**), where the first element added to the queue is the first one to be removed.





Types of Queues

Queue data structure can be classified into 3 types:



1. Simple Queue

A simple queue follows the FIFO (First In, First Out) principle.

- Insertion is allowed only at the rear (back).
- Deletion is allowed only from the front.
- Can be implemented using a linked list or a circular array.

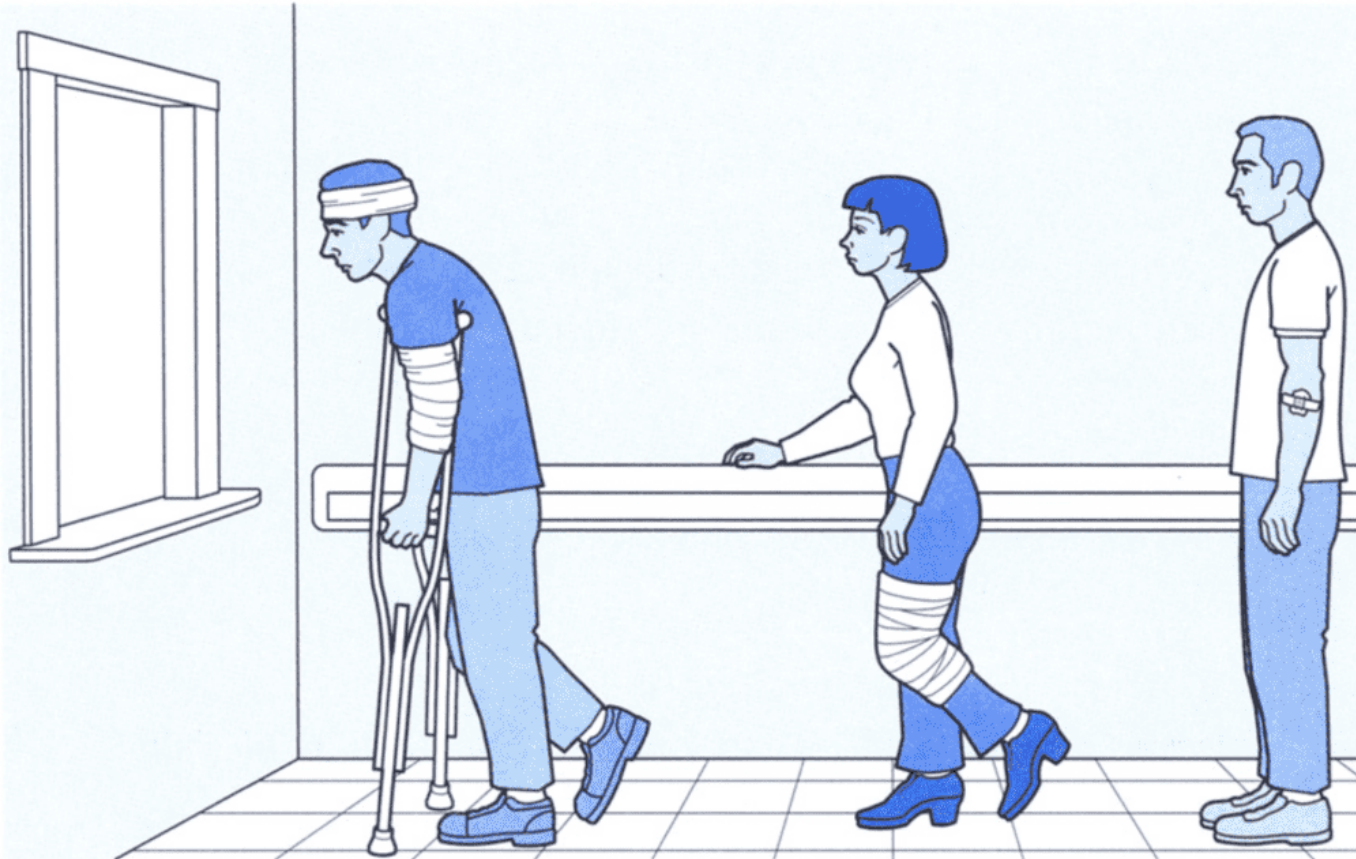
2. Double-Ended Queue (Deque)

In a deque, insertion and deletion can be performed from both ends.

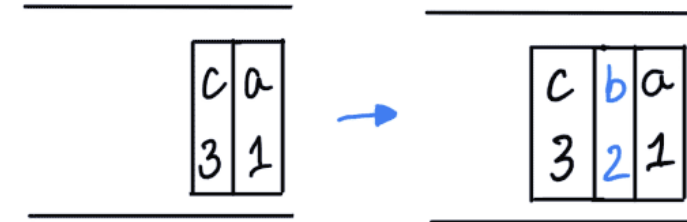
3. Priority Queue

A queue where each element is assigned a **priority**, and deletion always happens based on priority (not just position).

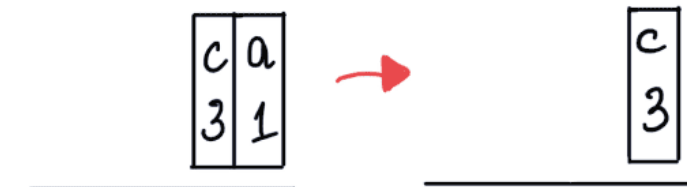




data priority
↓ ↓
ENQUEUE ['b', 2]



DEQUEUE [a, 1]



Queue Operations

- 1.Enqueue:** Adds an element to the end (rear) of the queue. If the queue is full, an overflow error occurs.
- 2.Dequeue:** Removes the element from the front of the queue. If the queue is empty, an underflow error occurs.
- 3.Peek/Front:** Returns the element at the front without removing it.
- 4.Size:** Returns the number of elements in the queue.
- 5.isEmpty:** Returns true if the queue is empty, otherwise false.
- 6.isFull:** Returns true if the queue is full, otherwise false.

✓ Example 1: Enqueue Operation (Insertion)

◆ Start with Empty Queue:

[]

◆ Operations:

Step	Operation	Action	Queue
1	Enqueue(10)	Insert 10	[10]
2	Enqueue(20)	Insert 20	[10, 20]
3	Enqueue(30)	Insert 30	[10, 20, 30]

◆ Final Queue:

[10, 20, 30]

✓ Example 2: Dequeue Operation (Deletion)

◆ Initial Queue:

[10, 20, 30]

◆ Operations:

Step	Operation	Action	Queue
1	Dequeue()	Remove 10	[20, 30]
2	Dequeue()	Remove 20	[30]

◆ Final Queue:

[30]

When a queue is implemented using an array, deletion (also called **dequeue**) is usually done from the **front**.

There are two common implementations:

1. Simple Linear Array Queue (Non-circular)

- Deletion is from the **front**.
- After deleting the front element, **all remaining elements must be shifted one position to the left** to maintain the queue order.

Example:

If queue = [10, 20, 30, 40]

After deleting 10, elements must shift:

[20, 30, 40, _]

👉 Shifting requires n operations

✅ Worst-case time complexity = $O(n)$

2. Circular Queue Implementation

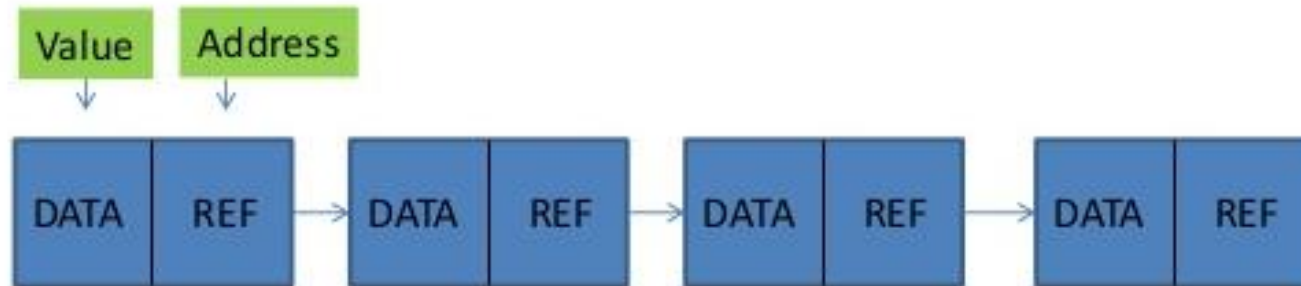
- Uses front and rear pointers.
- No shifting is needed.
- Deletion only moves the front pointer.

👉 Time complexity = $O(1)$



What are linked lists?

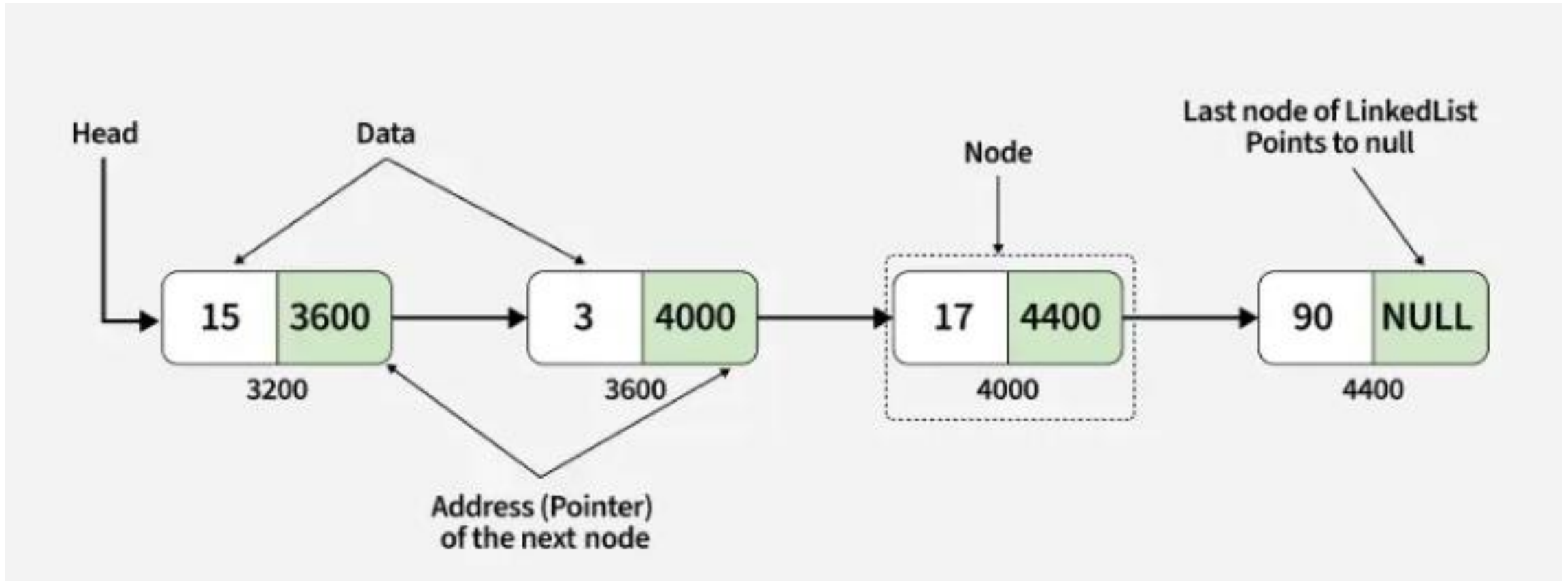
- Structure to store data similar to a train.



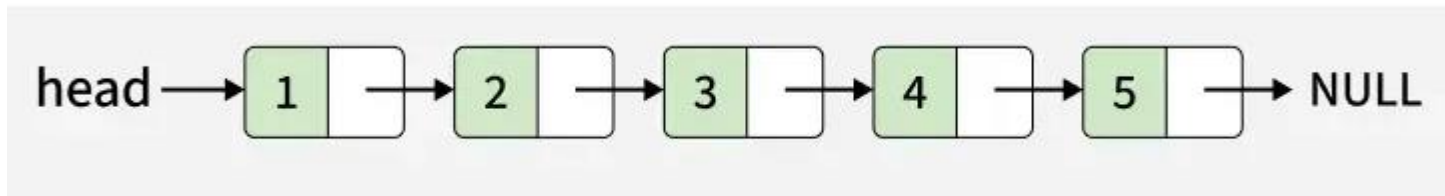
Linked List

A linked list is a type of linear data structure individual items are not necessarily at contiguous locations. The individual items are called nodes and connected with each other using links.

- A node contains two things first is data and second is a link that connects it with another node.
- The first node is called the head node and we can traverse the whole list using this head and next links.



- A **singly linked list** is a fundamental data structure, it consists of **nodes** where each node contains a **data** field and a **reference** to the next node in the linked list.
- The next of the last node is **null**, indicating the end of the list.
- Linked Lists support efficient insertion and deletion operations.



◆ Common Operations in a Linked List

A **linked list** is a dynamic data structure where elements (nodes) are connected using pointers.

1. Traversal (Display)

- Visiting each node one by one to access or display data.
- **Time Complexity:** $O(n)$

2. Insertion

Adding a new node into the linked list.

Types of Insertion:

- At the beginning
- At the end
- At a specific position
- After / before a given node

Time Complexity:

- Beginning $\rightarrow O(1)$
- End $\rightarrow O(n)$
- Specific position $\rightarrow O(n)$

3. Deletion

Removing a node from the linked list.

Types of Deletion:

- From beginning
- From end
- From a specific position
- By value (key)

Time Complexity: $O(n)$

4. Searching

Finding whether a given element exists in the list.

- **Time Complexity:** $O(n)$
-

5. Updating / Modification

Changing the value of an existing node.

- **Time Complexity:** $O(n)$

6. Sorting

Arranging elements in ascending or descending order.

- **Time Complexity:** $O(n \log n)^{**}$ (efficient algorithms)
 - Or $O(n^2)$ (bubble/selection sort)
-

7. Reversing

Reversing the direction of the linked list.

- **Time Complexity:** $O(n)$
-

8. Counting Nodes

Finding the total number of nodes.

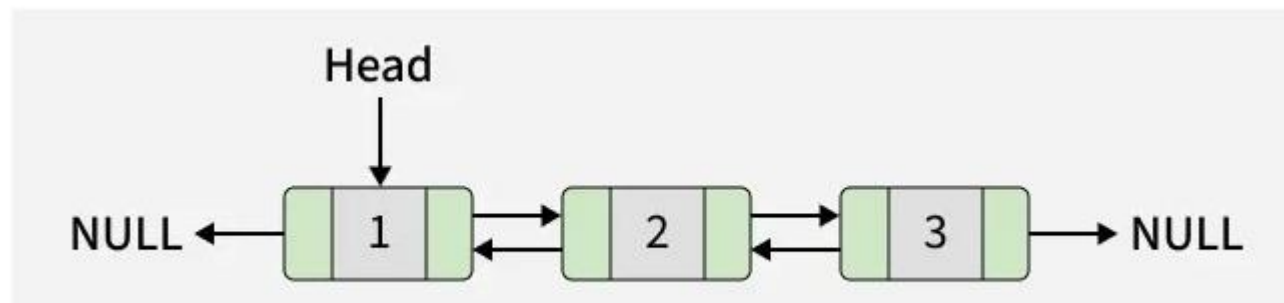
- **Time Complexity:** $O(n)$

◆ Summary Table

Operation	Purpose	Time Complexity
Traversal	Display nodes	$O(n)$
Insertion	Add node	$O(1)$ to $O(n)$
Deletion	Remove node	$O(n)$
Searching	Find element	$O(n)$
Updating	Modify data	$O(n)$
Sorting	Arrange order	$O(n \log n)$
Reversing	Reverse list	$O(n)$
Counting	Count nodes	$O(n)$

Doubly Linked List

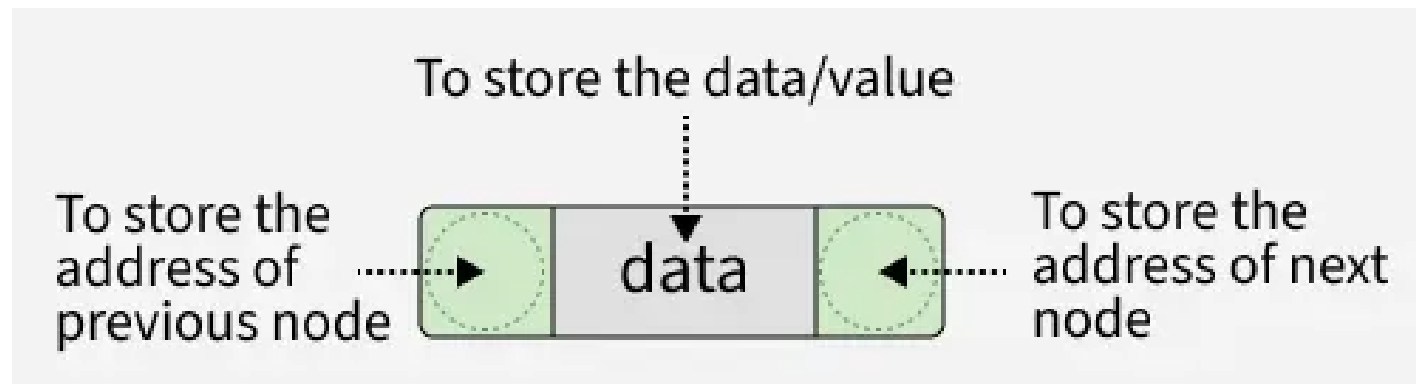
- A doubly linked list is a more complex data structure than a singly linked list, but it offers several advantages.
- The main advantage of a doubly linked list is that it allows for efficient traversal of the list in both directions.
- This is because each node in the list contains a pointer to the previous node and a pointer to the next node.
- This allows for quick and easy insertion and deletion of nodes from the list, as well as efficient traversal of the list in both directions.



Representation of Doubly Linked List in Data Structure

In a data structure, a doubly linked list is represented using nodes that have three fields:

1. Data
2. A pointer to the next node (**next**)
3. A pointer to the previous node (**prev**)



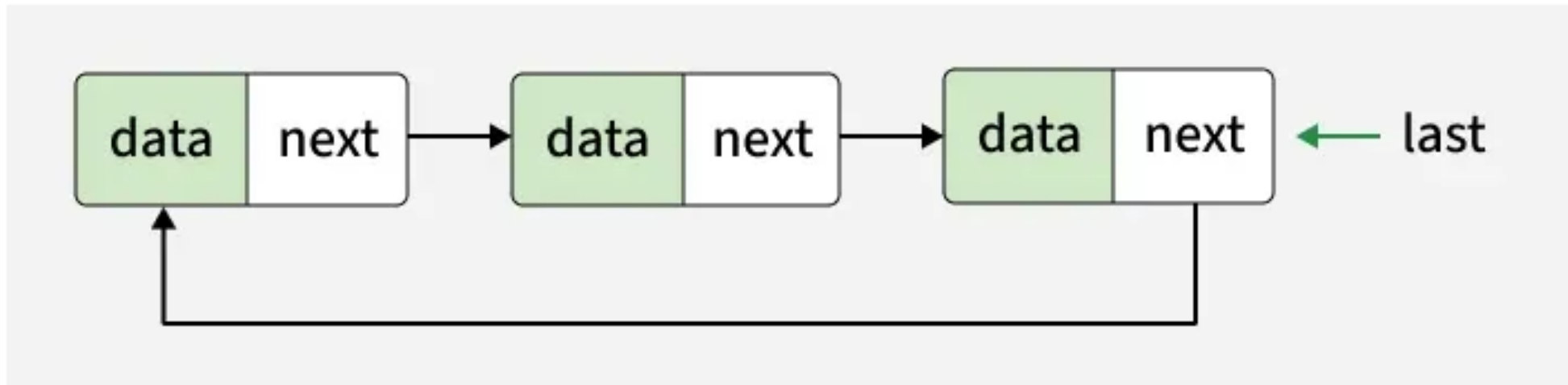
Circular Linked List

A circular linked list is a data structure where the last node points back to the first node, forming a closed loop.

- **Structure:** All nodes are connected in a circle, enabling continuous traversal without encountering NULL.
- **Difference from Regular Linked List:** In a regular linked list, the last node points to NULL, whereas in a circular linked list, it points to the first node.
- **Uses:** Ideal for tasks like scheduling and managing playlists, where smooth and repeated.

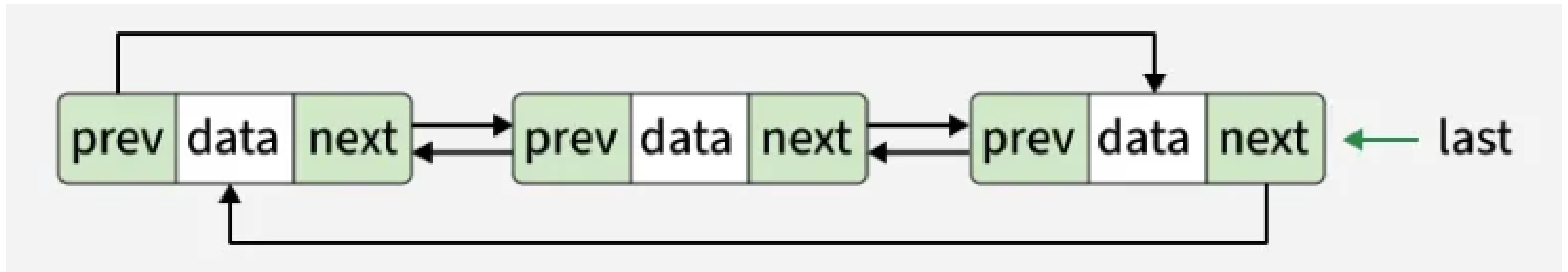
1. Circular Singly Linked List

In Circular Singly Linked List, each node has just one pointer called the "next" pointer. The next pointer of the last node points back to the first node and this results in forming a circle. In this type of Linked list, we can only move through the list in one direction.



2. Circular Doubly Linked List:

In circular doubly linked list, each node has two pointers prev and next, similar to doubly linked list. The prev pointer points to the previous node and the next points to the next node. Here, in addition to the last node storing the address of the first node, the first node will also store the address of the last node.



Tree

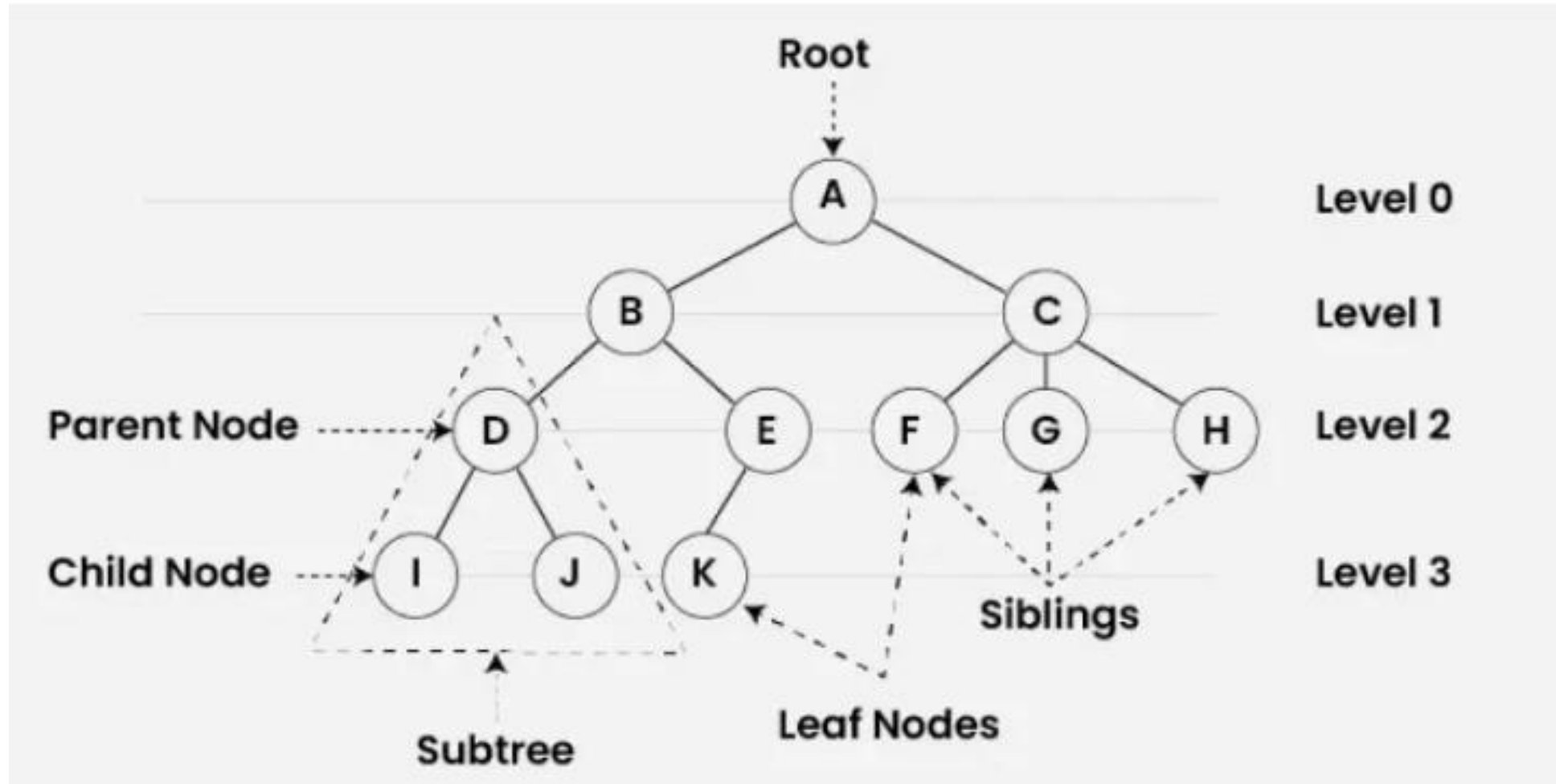
A **tree** is a **non-linear hierarchical data structure** that consists of **nodes** connected by **edges**. It starts with a **root node**, and every node can have **zero or more child nodes**.

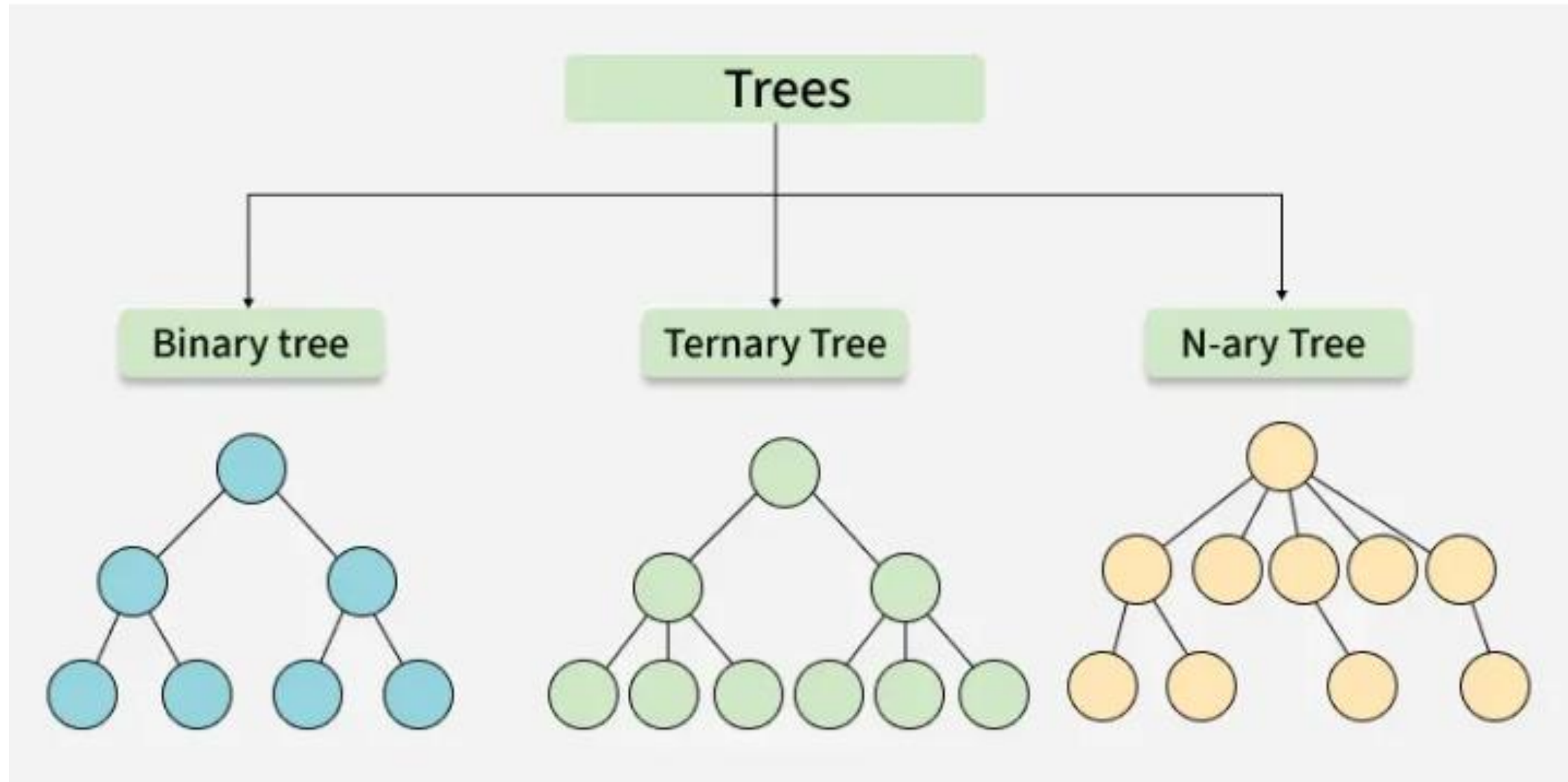
There are **no cycles** in a tree.

Trees are used to represent **hierarchical relationships**, such as:

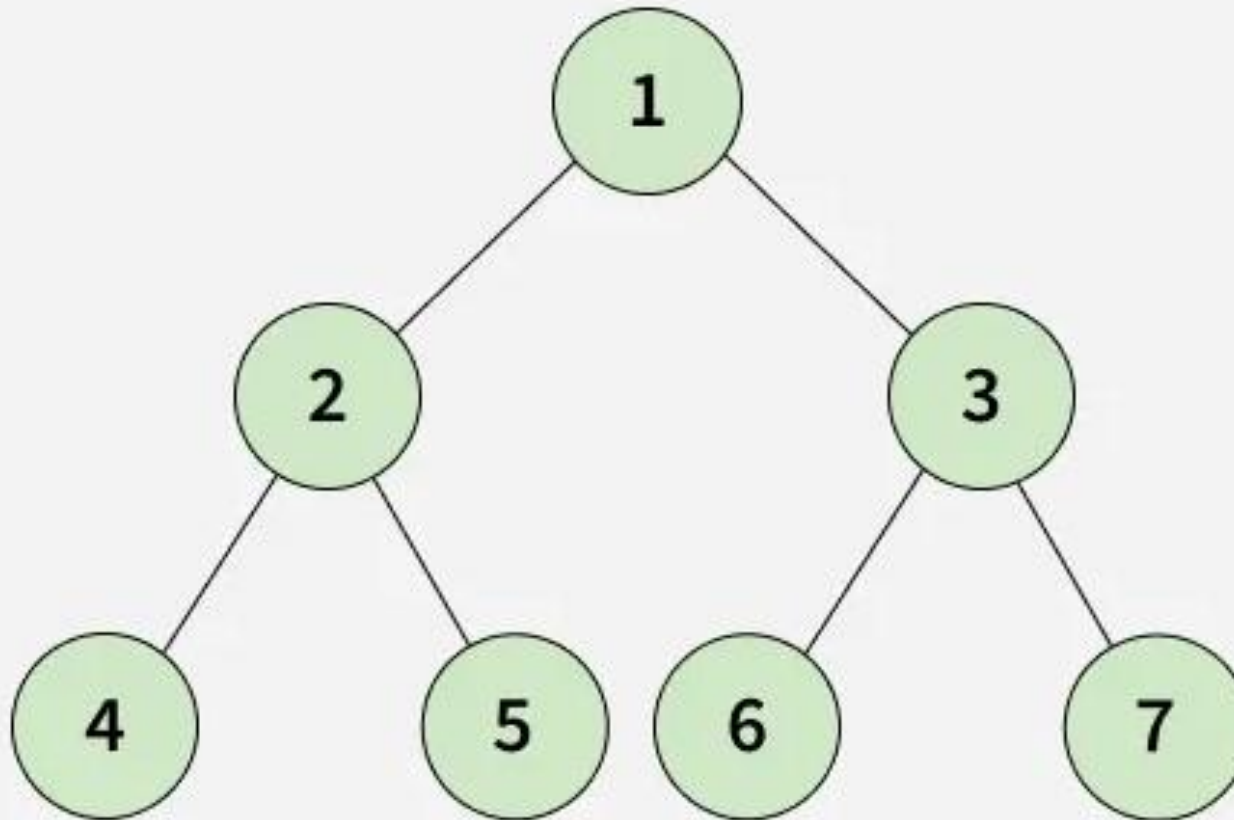
- File and folder structure in a computer
- Organization hierarchy
- Family tree
- HTML document structure

- **Root:** Topmost node of the tree
- **Parent:** A node that has children
- **Child:** A node connected below another node
- **Leaf (External node):** Node with no children
- **Edge:** Connection between two nodes
- **Subtree:** A tree formed from any node

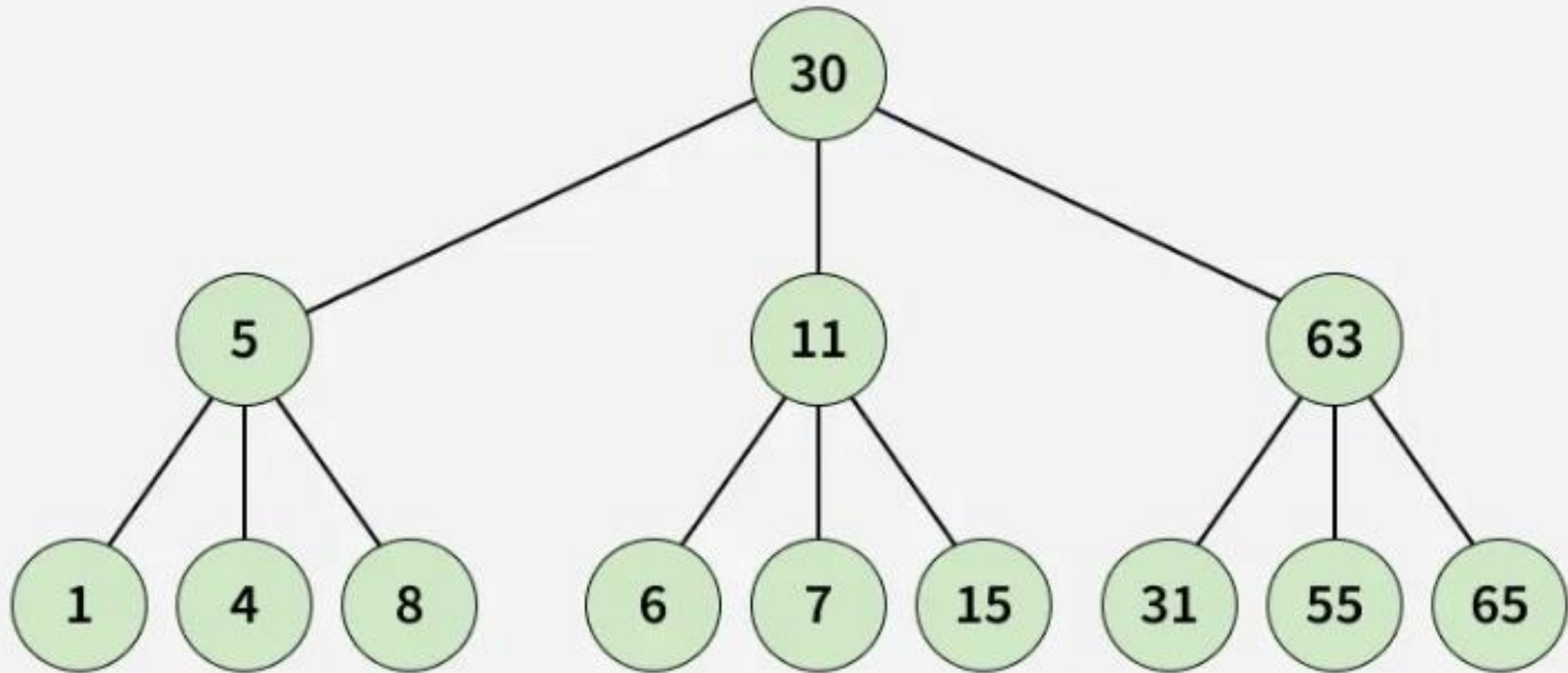




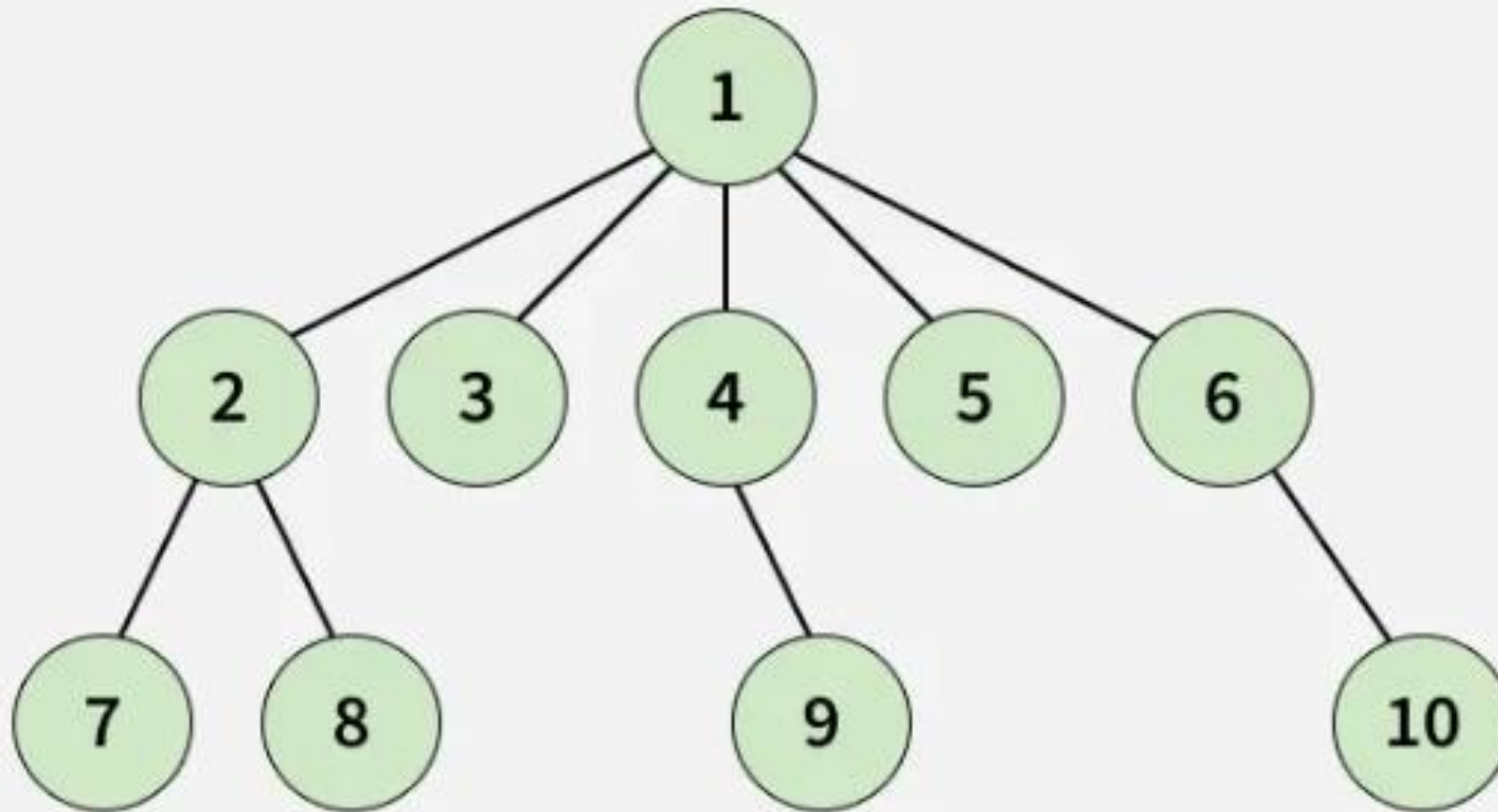
Binary Tree



Ternary Tree



N-ary Tree

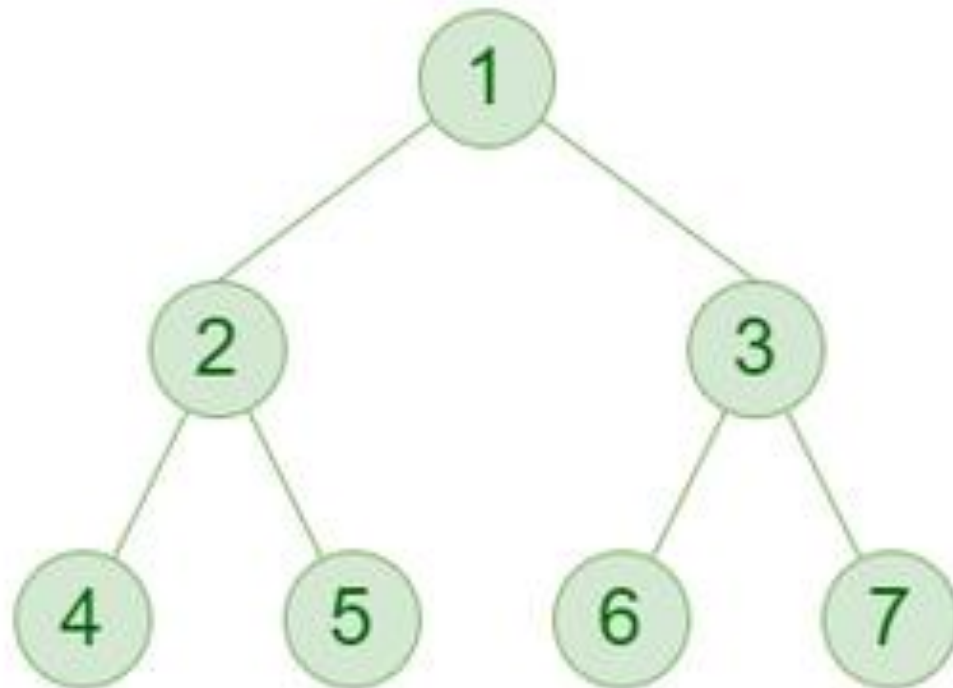


Binary Search Tree (BST)

A **Binary Search Tree (BST)** is a type of **binary tree** in which:

- The **left subtree** of a node contains **only values smaller** than the node.
- The **right subtree** of a node contains **only values greater** than the node.
- Both left and right subtrees are also **binary search trees**.

Binary Tree



Level (i)

0

$$(2^0 = 1)$$

1

$$(2^1 = 2)$$

2

$$(2^2 = 4)$$

3

$$(2^3 = 8)$$

i

$$(2^i)$$

Operations in BST

1. **Insertion** – Insert node at proper position.
2. **Deletion** – Remove a node and rearrange tree.
3. **Searching** – Find an element efficiently.
4. **Traversal** – Inorder, Preorder, Postorder.

Time Complexity

Operation	Average Case	Worst Case
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

Draw a BST for the following key values 15, 10, 20, 8, 12, 25, 17, 11.

Step-by-step Insertion

1. **15** → Root

2. **10** → Left of 15

3. **20** → Right of 15

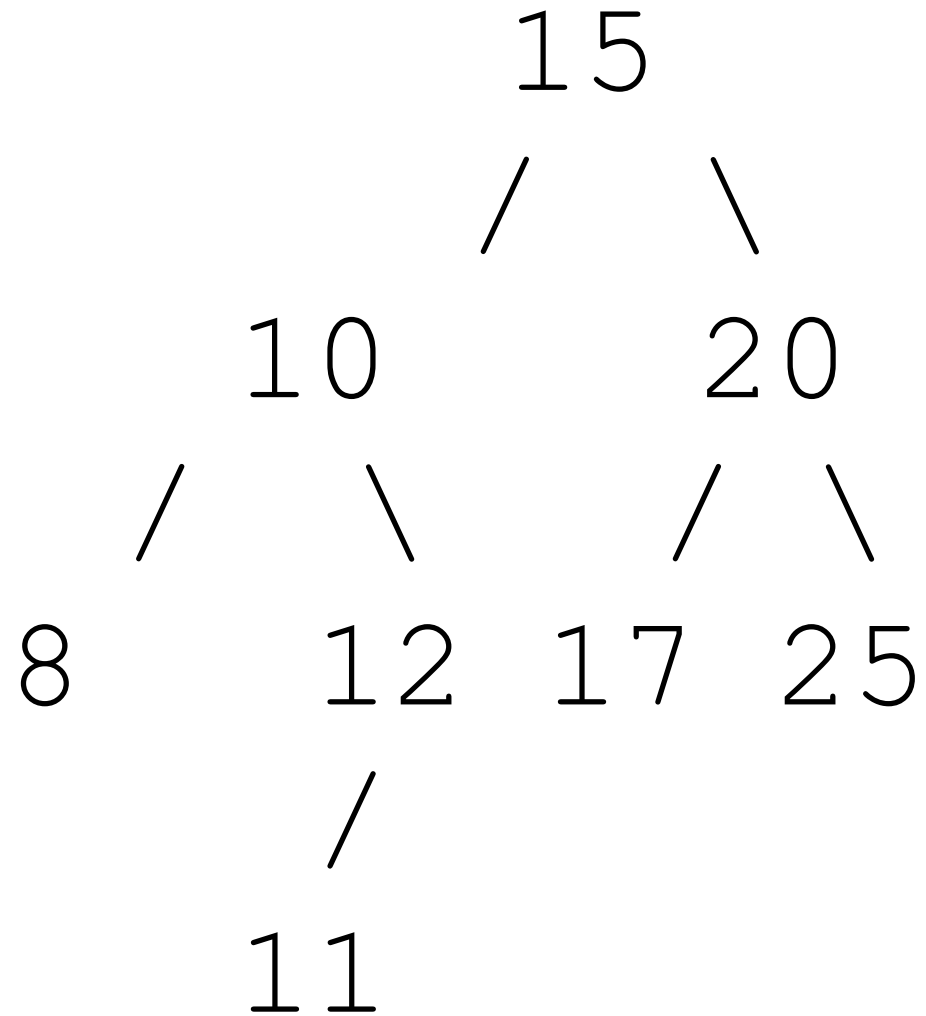
4. **8** → Left of 10

5. **12** → Right of 10

6. **25** → Right of 20

7. **17** → Left of 20

8. **11** → Left of 12



Binary Search Tree (BST) Construction

Given Data (Insert in order):

200, 150, 350, 100, 70, 110, 250, 500, 400, 550, 450

Step-by-step Insertion

1. **200** → Root

2. **150** → Left of 200

3. **350** → Right of 200

4. **100** → Left of 150

5. **70** → Left of 100

6. **110** → Right of 100

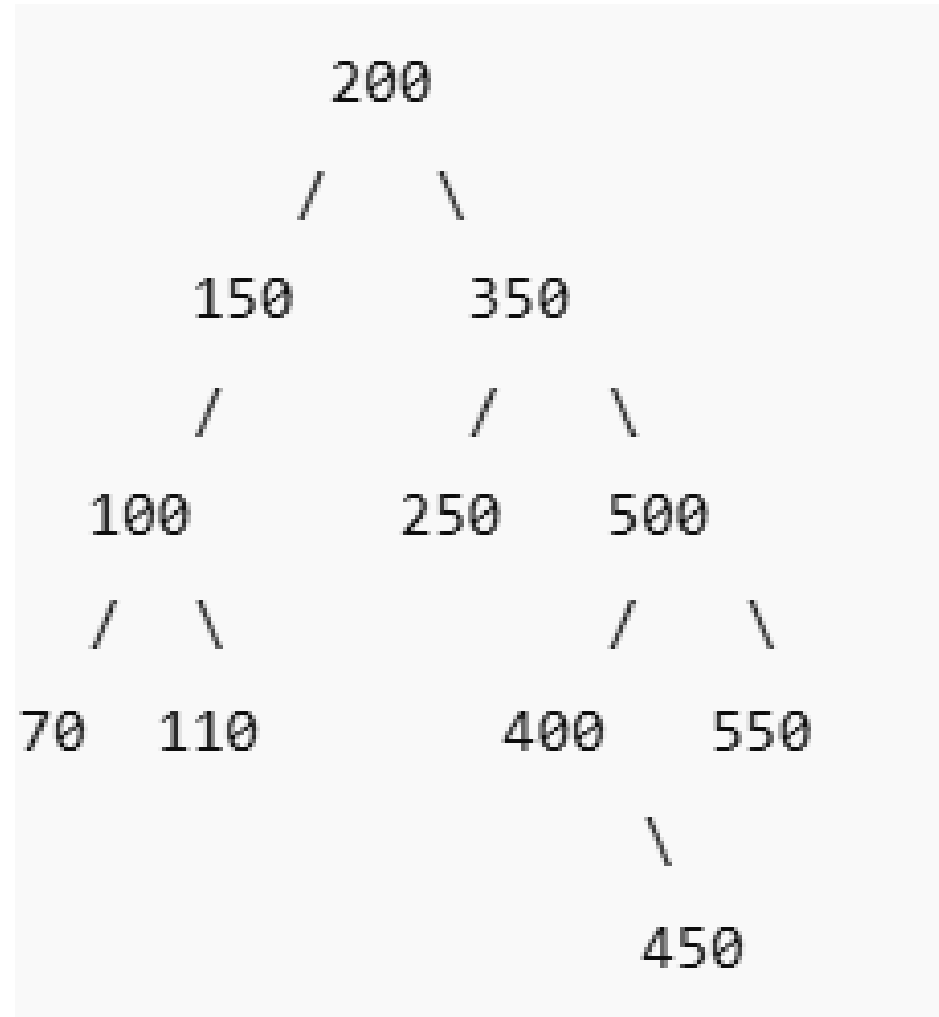
7. **250** → Left of 350

8. **500** → Right of 350

9. **400** → Left of 500

10. **550** → Right of 500

11. **450** → Right of 400

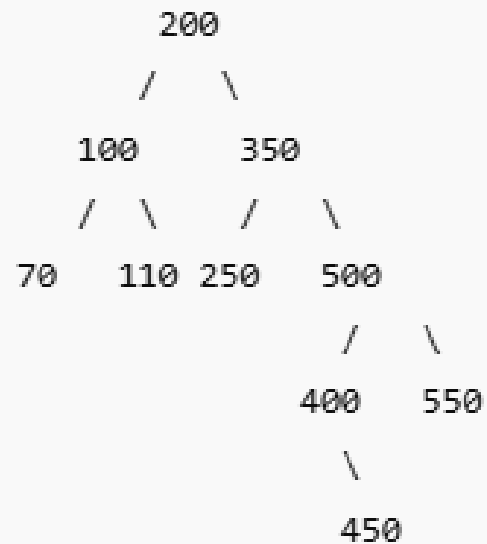


Deletion Order:

150, 500, 450, 200, 110

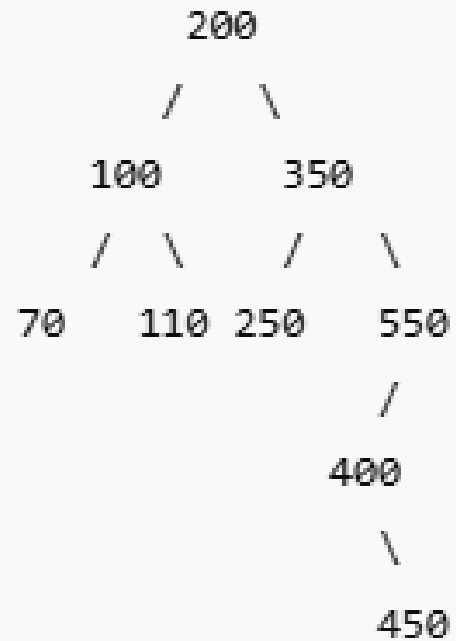
Step 1: Delete 150

Node 150 has one child (100) → Replace 150 with 100.



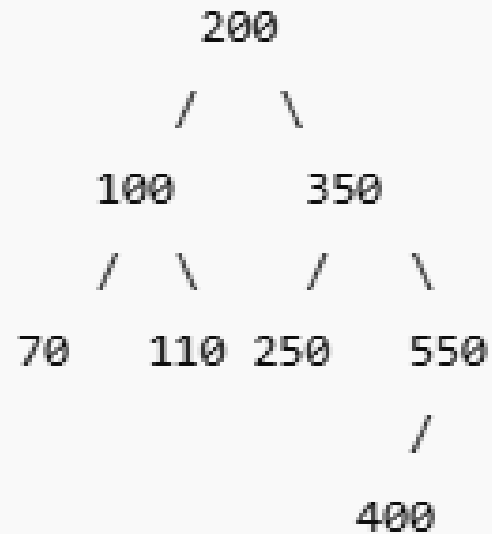
✓ Step 2: Delete 500

Node 500 has two children (400, 550) → Replace with inorder successor (550).



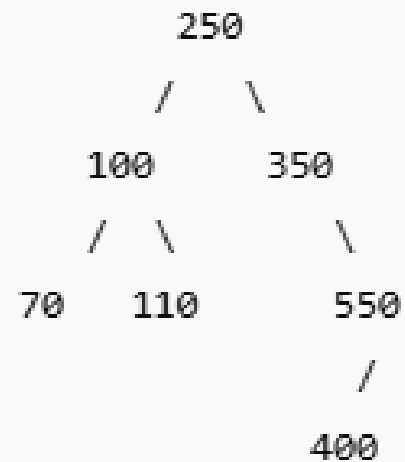
Step 3: Delete 450

Node 450 is a leaf node → Simply remove it.



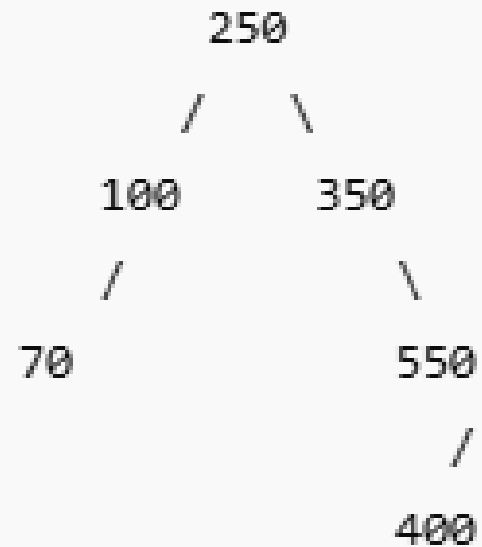
✓ Step 4: Delete 200

Node 200 has two children (100, 350) → Replace with inorder successor = 250.

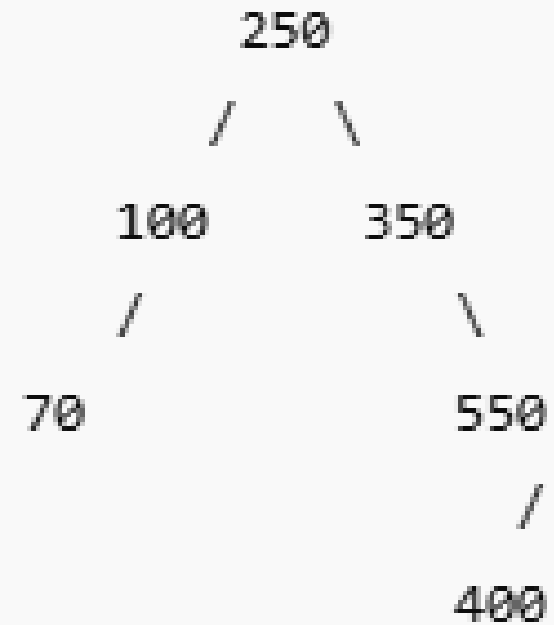


✓ Step 5: Delete 110

Node 110 is a leaf node → Simply remove.

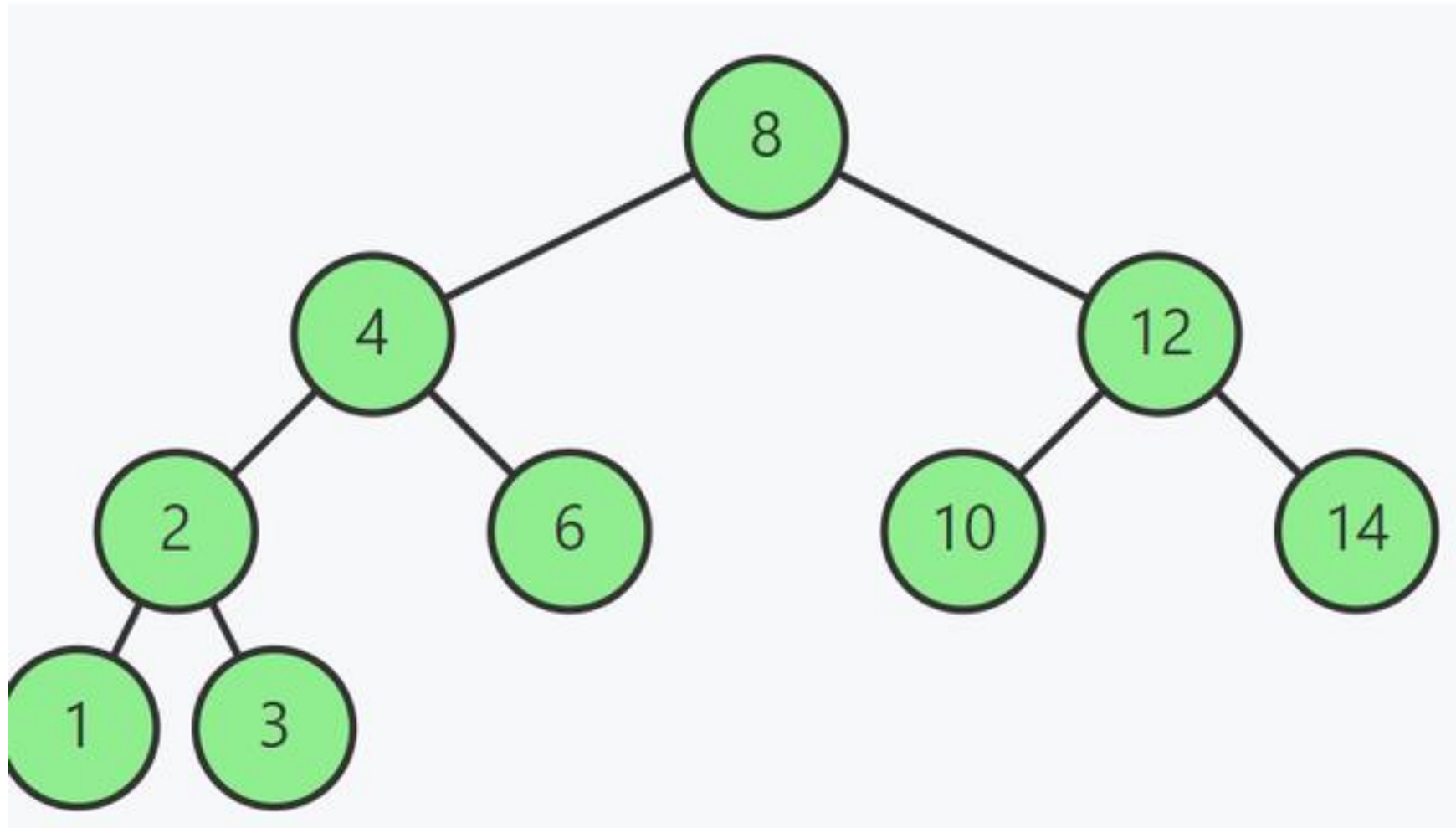


Final BST After All Deletions



Balanced Binary Tree

A **Balanced Binary Tree** is a binary tree in which the **height difference between the left and right subtrees of every node is at most 1**. This ensures that the tree remains **balanced**, providing **efficient operations**.



Why Balanced Binary Tree is Important?

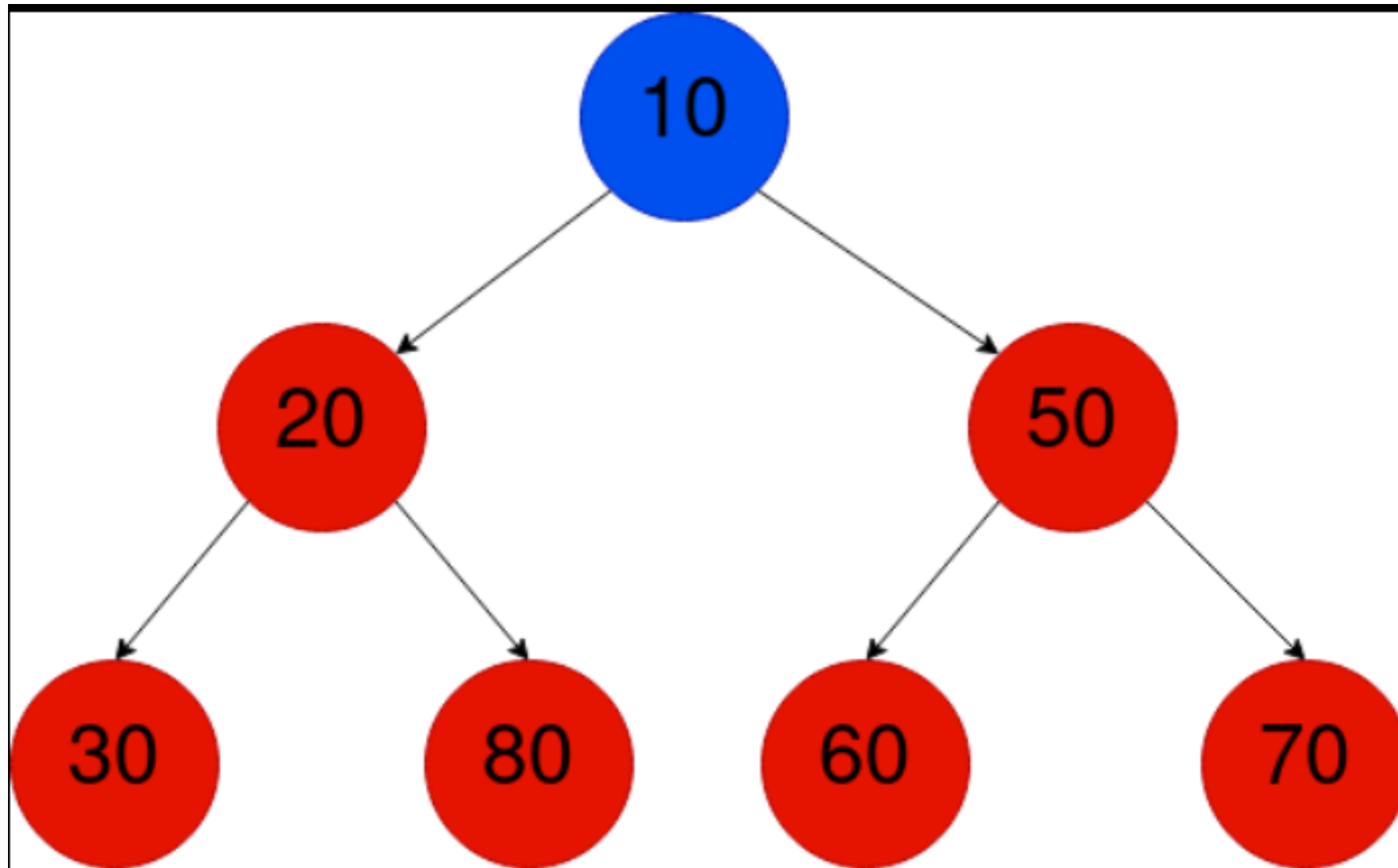
- Prevents the tree from becoming skewed
- Guarantees fast searching, insertion, and deletion
- Maintains $O(\log n)$ time complexity

Time Complexity

Operation	Time
Search	$O(\log n)$
Insert	$O(\log n)$
Delete	$O(\log n)$

A **Heap** is a **complete binary tree** that satisfies the **heap property**:

- **Max Heap:** Parent node value $\geq$ its children
- **Min Heap:** Parent node value $\leq$ its children
- Heaps are mainly used to implement **Priority Queues**.

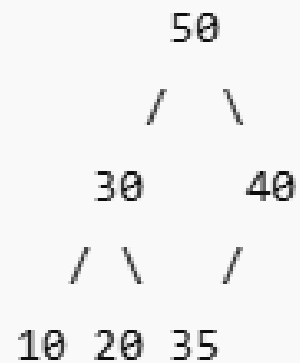


Types of Heap

1. Max Heap

The largest element is always at the root.

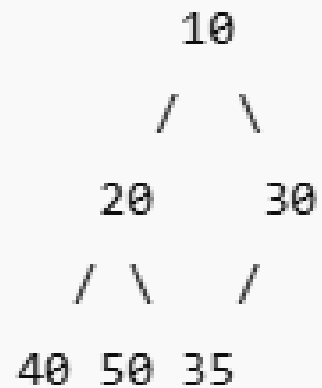
Example:



2. Min Heap

The smallest element is always at the root.

Example:



Given

Preorder traversal of BST:

15, 10, 12, 11, 20, 18, 16, 19

We must construct the BST from preorder and then find the **postorder traversal**.

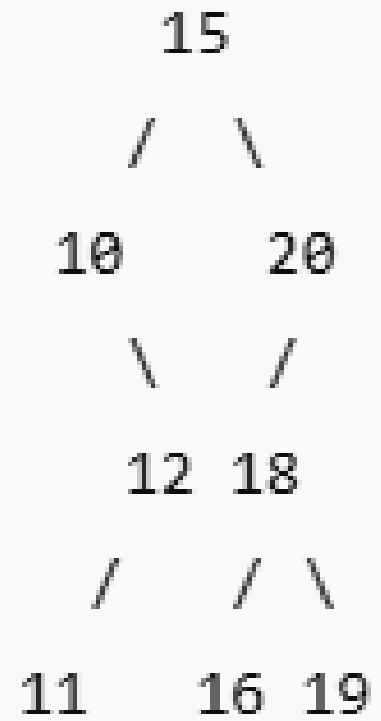
Step 1: Construct the BST

Rule of BST:

- Left subtree → smaller values
- Right subtree → larger values

Insert nodes one by one (as per preorder):

1. 15 → Root
2. 10 → Left of 15
3. 12 → Right of 10
4. 11 → Left of 12
5. 20 → Right of 15
6. 18 → Left of 20
7. 16 → Left of 18
8. 19 → Right of 18



Step 2: Postorder Traversal Rule

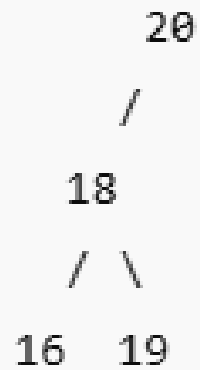
Postorder = Left → Right → Root

Left Subtree of 15 (root = 10):

```
10
 \
 12
 /
11
```

Postorder → 11, 12, 10

Right Subtree of 15 (root = 20):



Postorder → 16, 19, 18, 20

Now Root → 15

Final Postorder Traversal

11, 12, 10, 16, 19, 18, 20, 15

 **Answer:**

Postorder traversal =

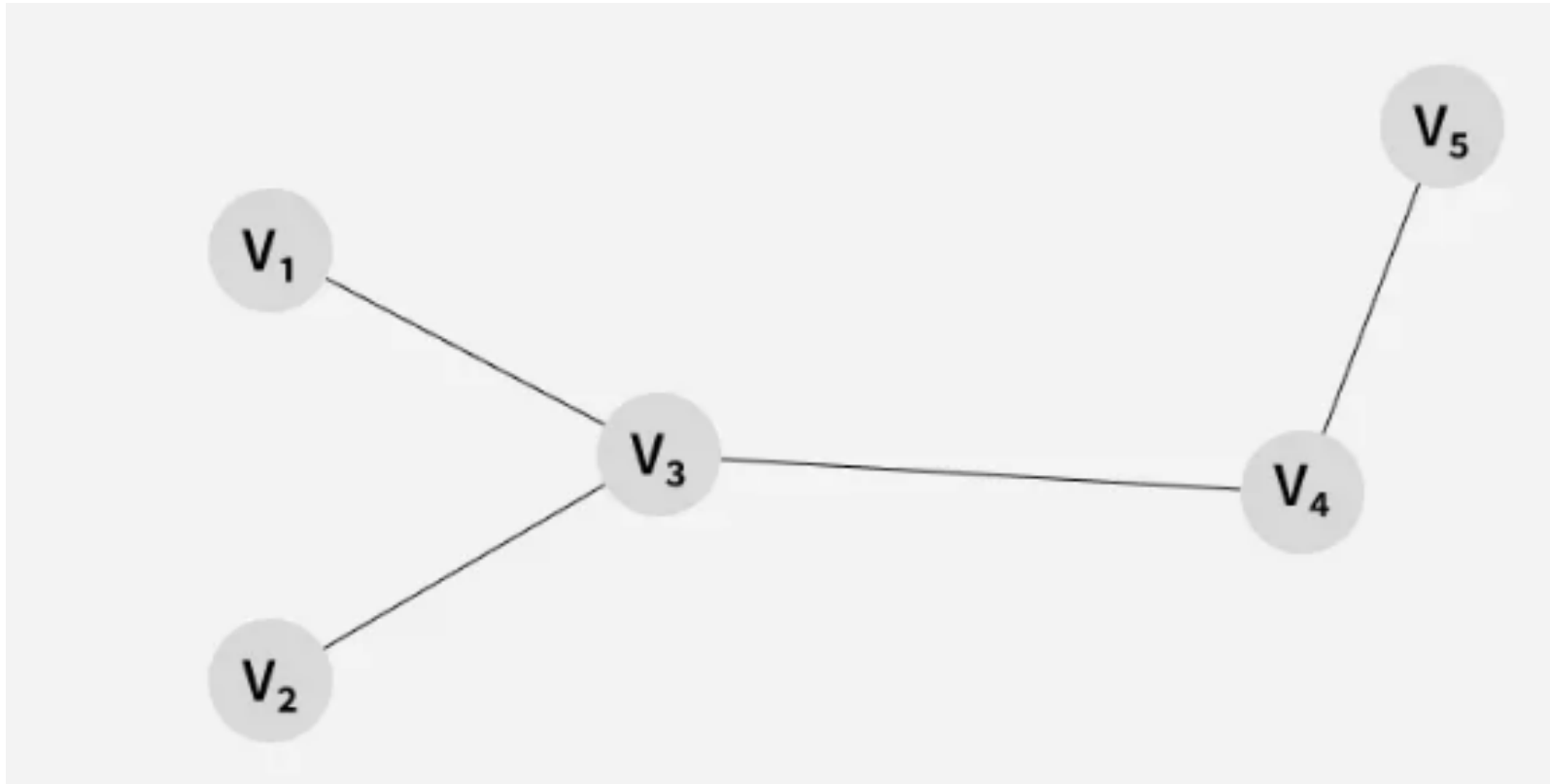
 11, 12, 10, 16, 19, 18, 20, 15

Graph

- Graph is a non-linear data structure like tree data structure.
- A Graph is composed of a set of vertices(**V**) and a set of edges(**E**).
- The vertices are connected with each other through edges.
- **Vertices (or nodes)**: The points in the graph representing entities.
- **Edges**: The lines connecting pair of vertices, representing relationships or interactions.

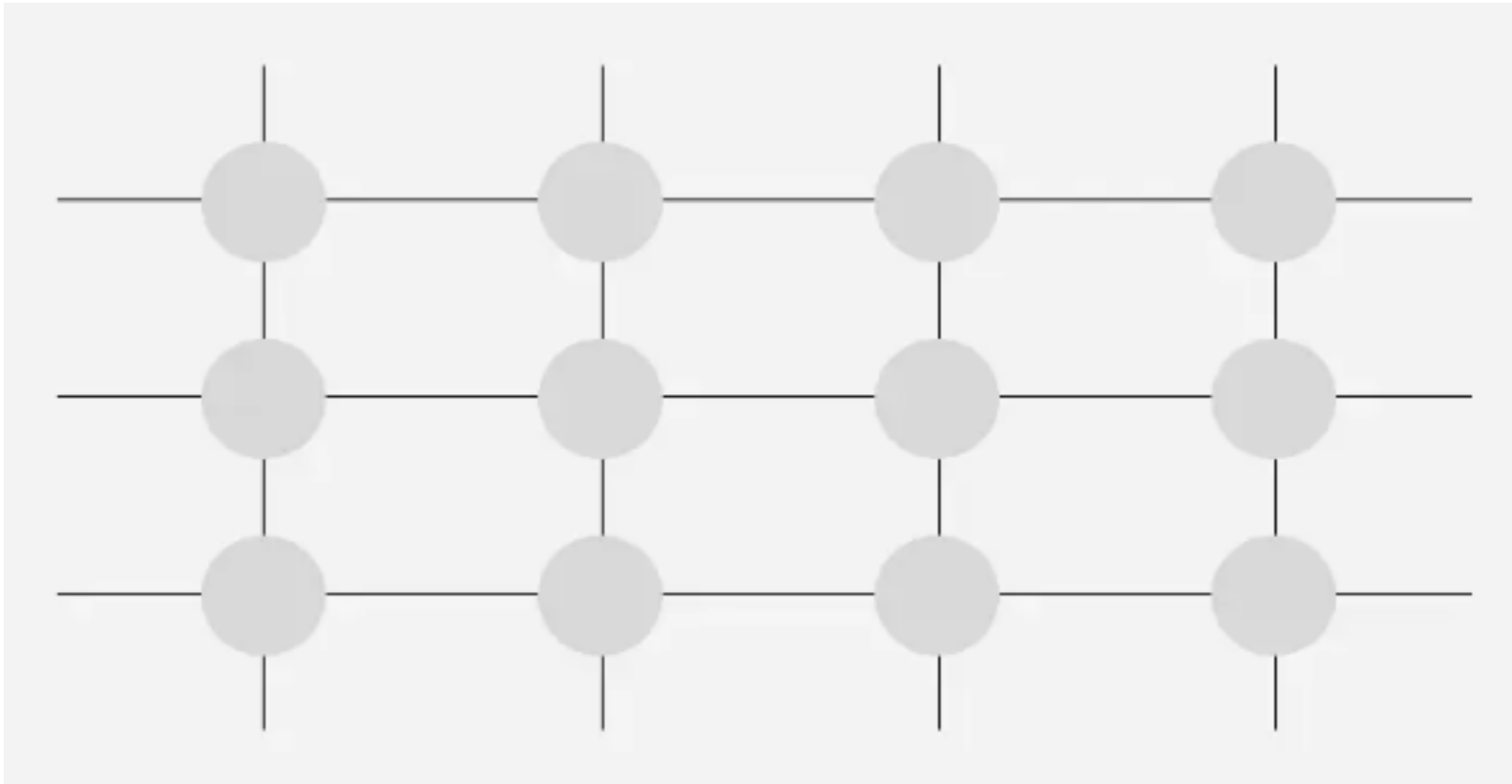
Finite Graphs

A finite graph is a graph with a finite number of vertices and edges. In other words, both the number of vertices and the number of edges in a finite graph are limited and can be counted.



Infinite Graph:

A graph is called an infinite graph if it has an infinite number of vertices and an infinite number of edges. Unlike finite graphs, which have a fixed number of nodes and connections, infinite graphs extend indefinitely.



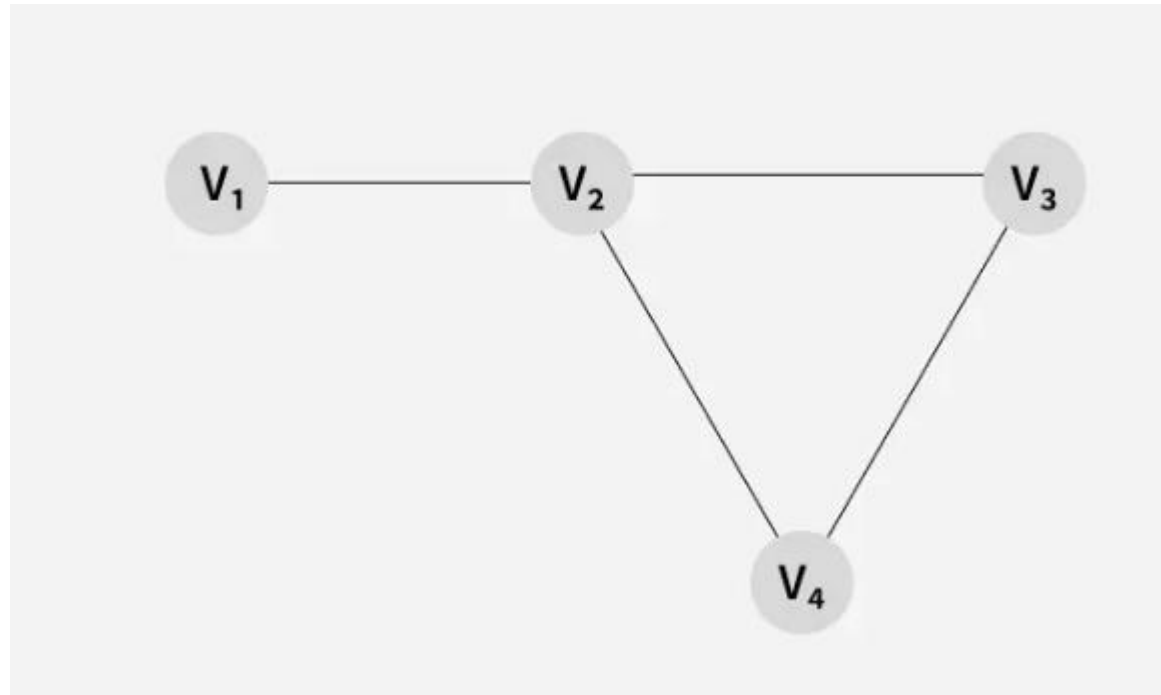
Trivial Graph

A graph is said to be trivial if a finite graph contains only one vertex and no edge. It is also known as a singleton graph or a single vertex graph



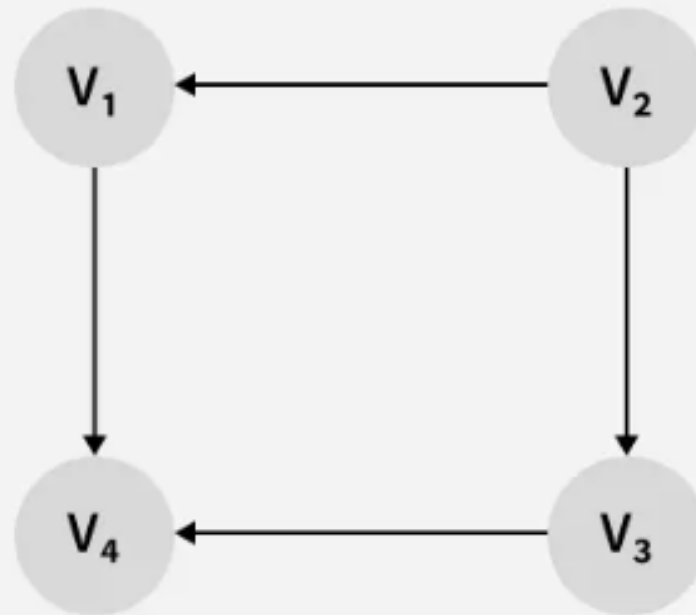
Simple Graph

A simple graph is a graph that does not contain more than one edge between the pair of vertices. A simple railway track connecting different cities is an example of a simple graph.



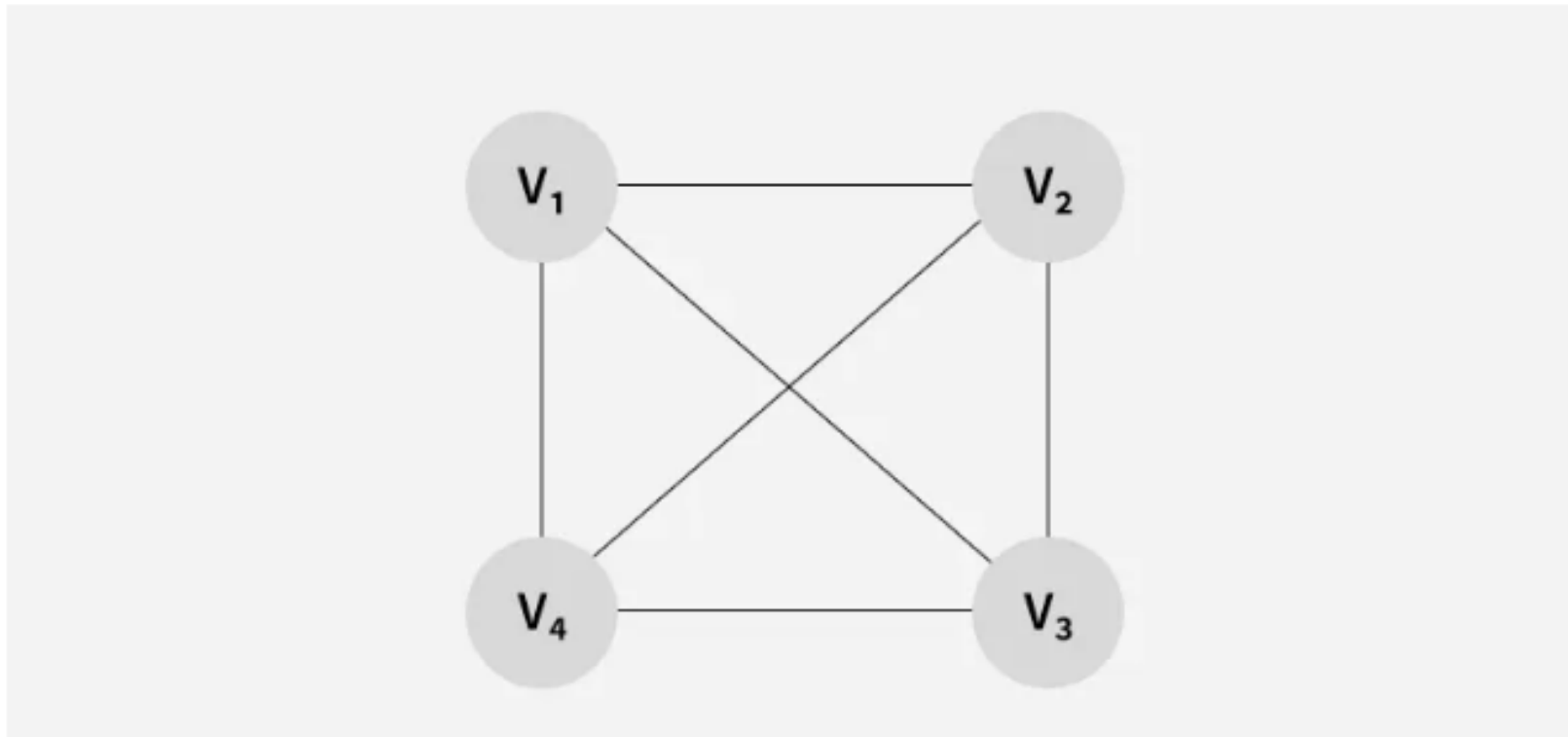
Directed Graphs:

A graph in which edges have a direction, i.e., the edges have arrows indicating the direction of traversal.



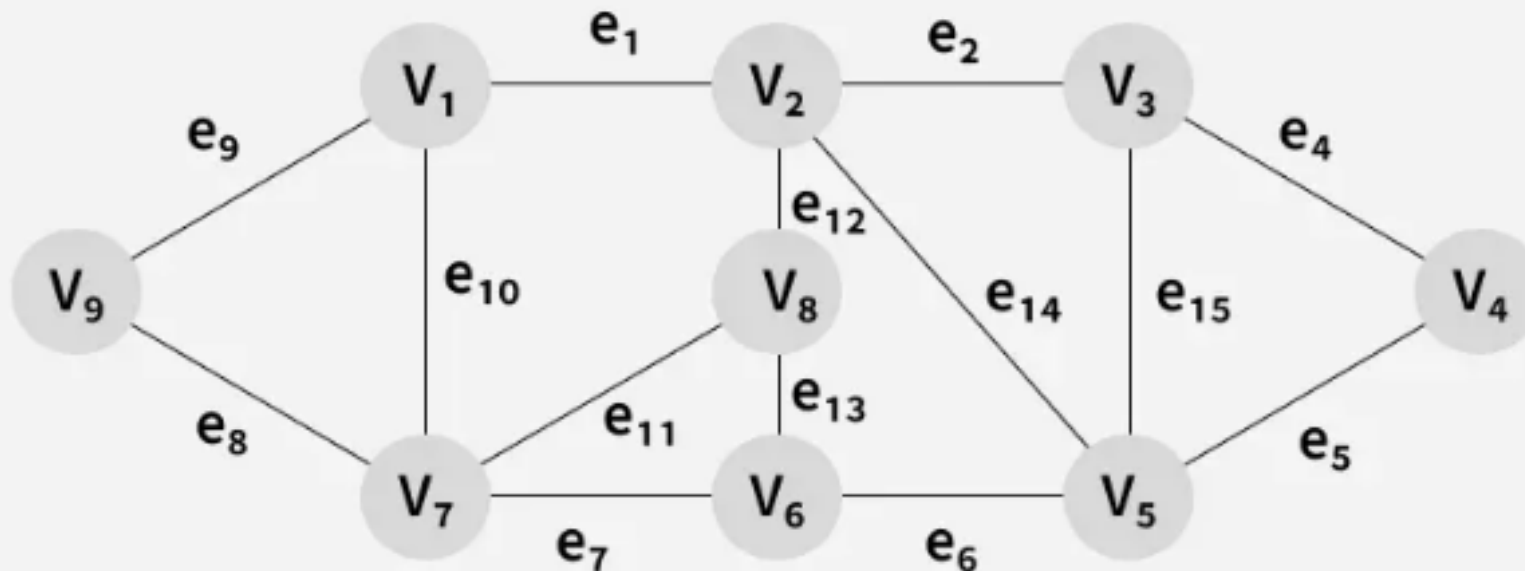
Undirected Graphs

An undirected graph is a graph where edges do not have a specific direction, meaning connections go both ways. If two places are connected, you can travel in either direction. Examples include friendships on social media and two-way roads.



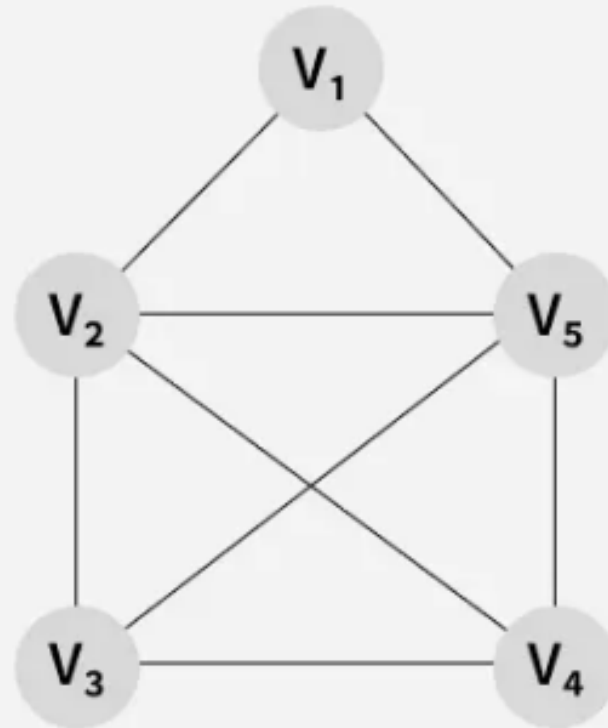
Weighted Graphs

A **weighted graph** is a graph where each edge has a number (weight) that represents distance, cost, or time. These graphs help find the shortest or cheapest paths. Examples include Google Maps, airline routes, and delivery networks.



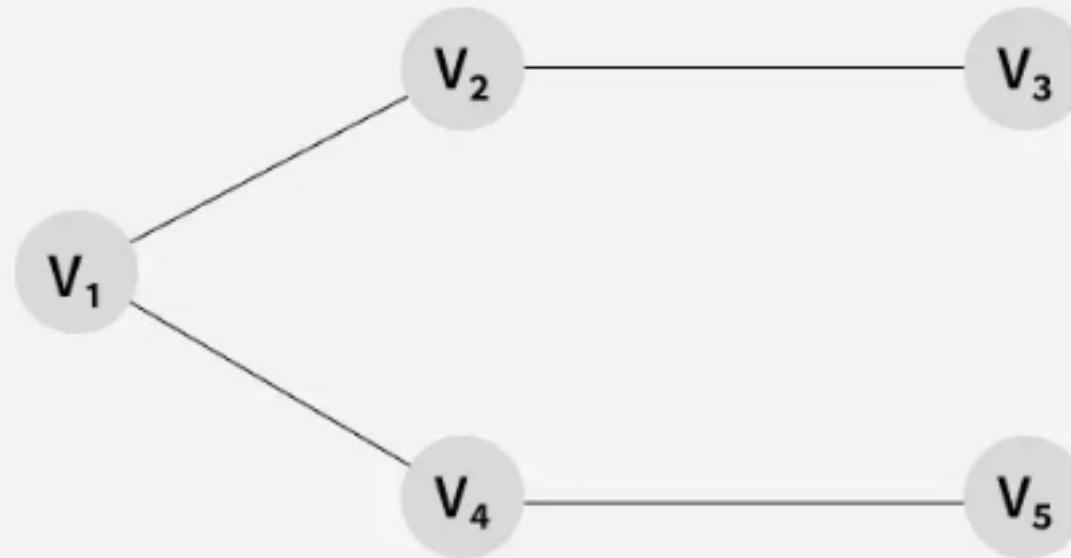
Unweighted Graphs

An **unweighted graph** is a graph where all edges are treated equally, with no extra values like distance or cost. It simply shows connections between points. Examples include basic social networks and metro maps without travel times.



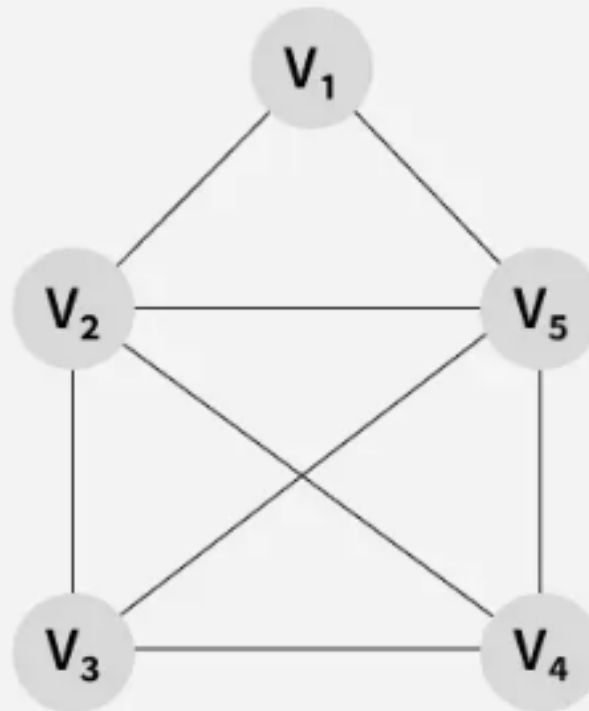
Sparse Graphs

A graph with relatively few edges compared to the number of vertices. Example: A chemical reaction graph where each vertex represents a chemical compound and each edge represents a reaction between two compounds.



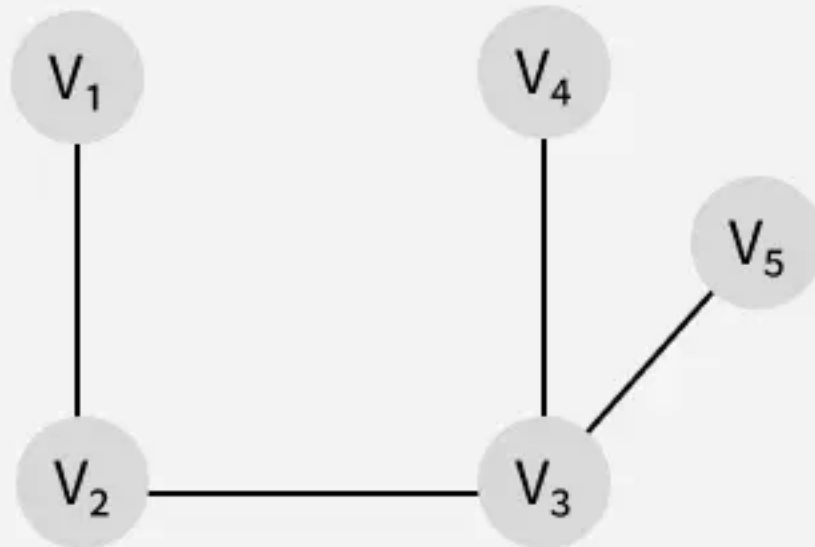
Dense Graphs

A graph with many edges compared to the number of vertices. Example: A social network graph where each vertex represents a person and each edge represents a friendship.

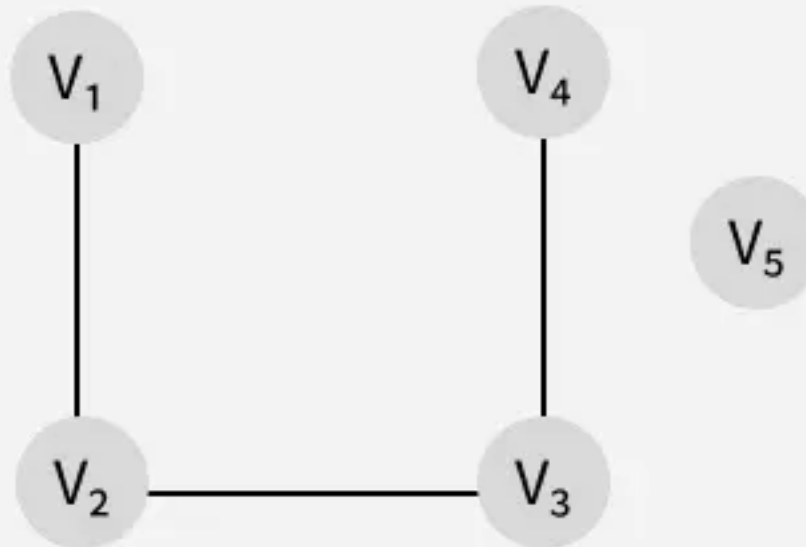


Connected or Disconnected Graph

Graph is said to be connected if there exists at least one path between each and every pair of vertices in graph G , otherwise, it is disconnected. A null graph with n vertices is a disconnected graph consisting of n components. Each component consists of one vertex and no edge.



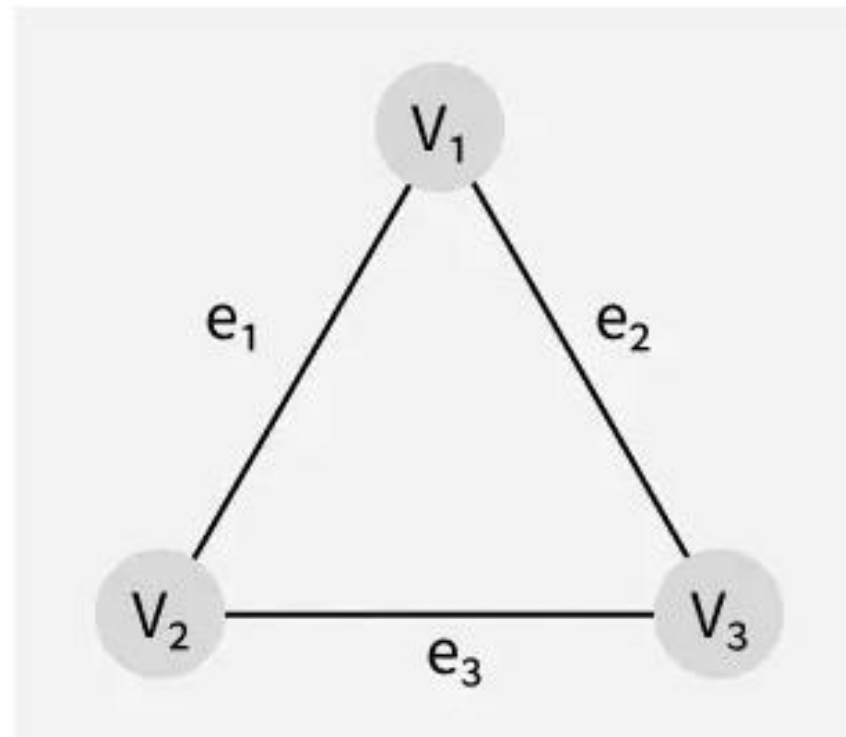
Connected Graph



Disconnected Graph

Cyclic Graph

A graph G consisting of n vertices and $n \geq 3$ that is $V_1, V_2, V_3, \dots, V_n$ and edges $(V_1, V_2), (V_2, V_3), (V_3, V_4), \dots, (V_n, V_1)$ are called cyclic graph.



BFS (Breadth-First Search)?

BFS (Breadth-First Search) is a **graph traversal algorithm** used to explore all the vertices of a graph **level by level**.

It visits:

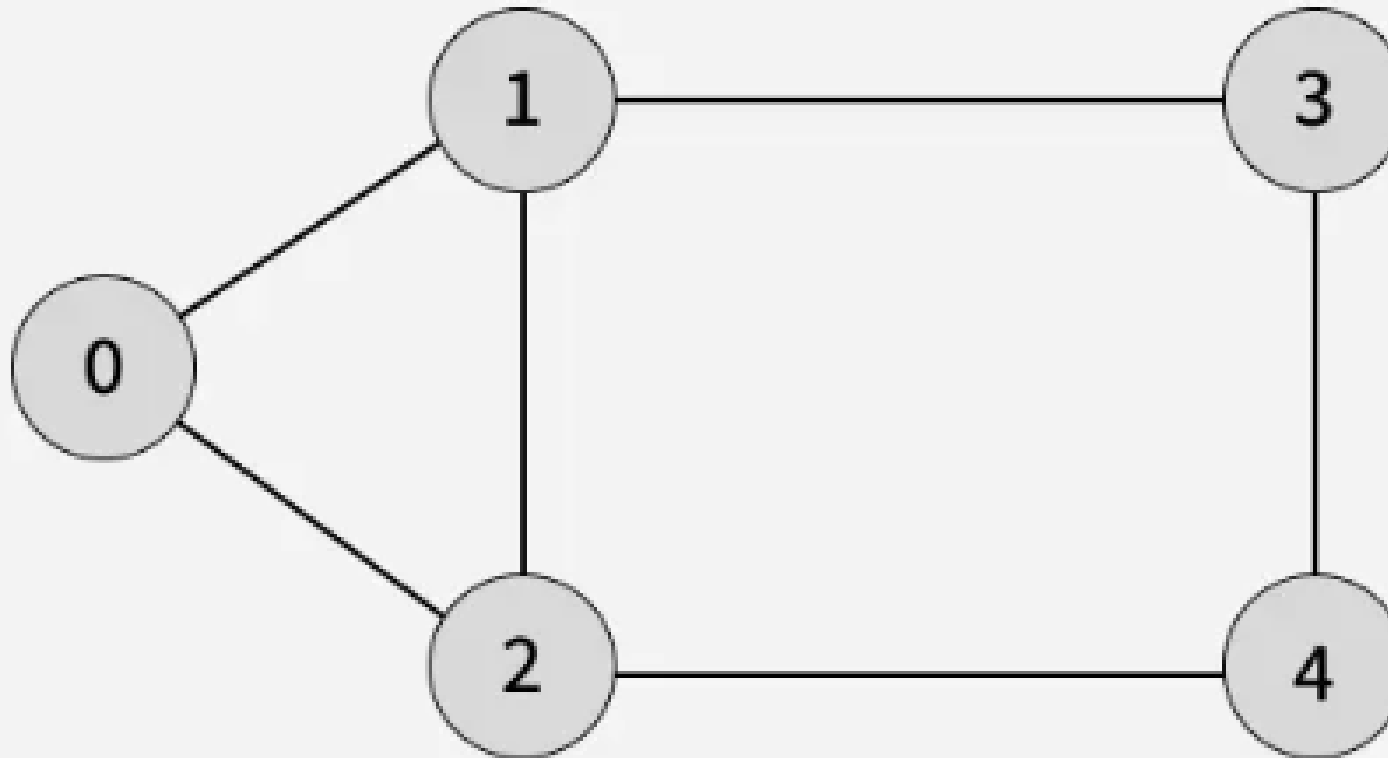
- First → the **starting node**
- Then → **all its immediate neighbors**
- Then → neighbors of those neighbors
- ...and so on.

It uses a **QUEUE (FIFO – First In First Out)** data structure.

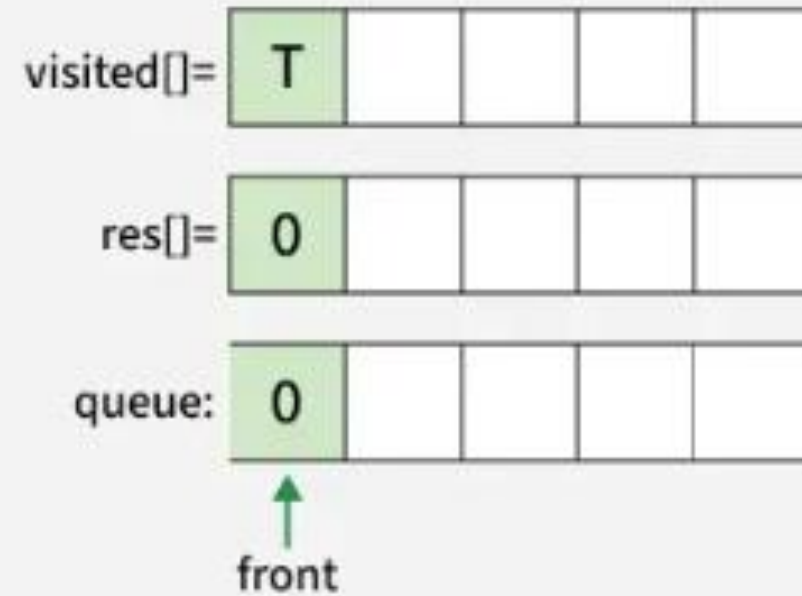
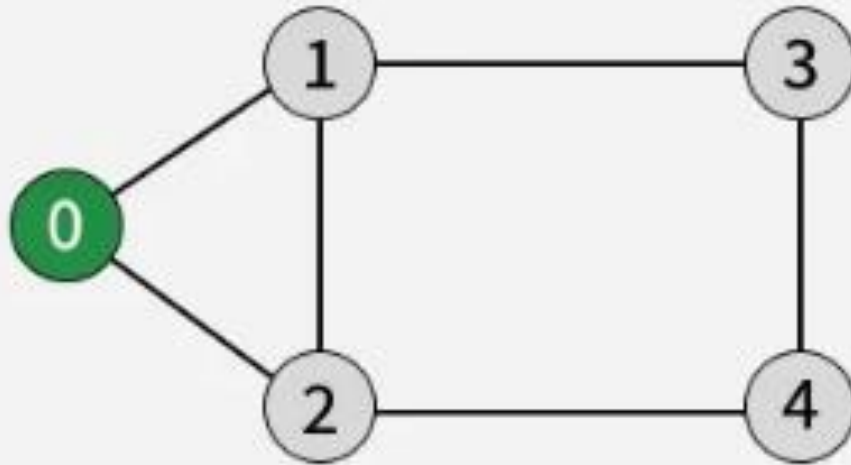
How BFS Works (Steps):

1. Start from a **source node**
2. Mark it as **visited**
3. Insert it into a **queue**
4. While queue is not empty:
 1. Remove front element
 2. Visit all its **unvisited neighbors**
 3. Add them to the queue

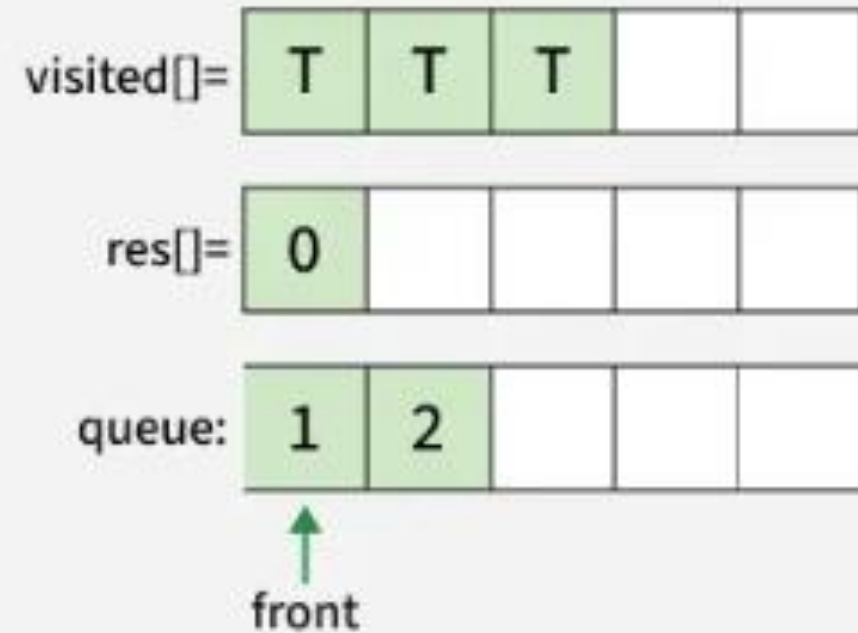
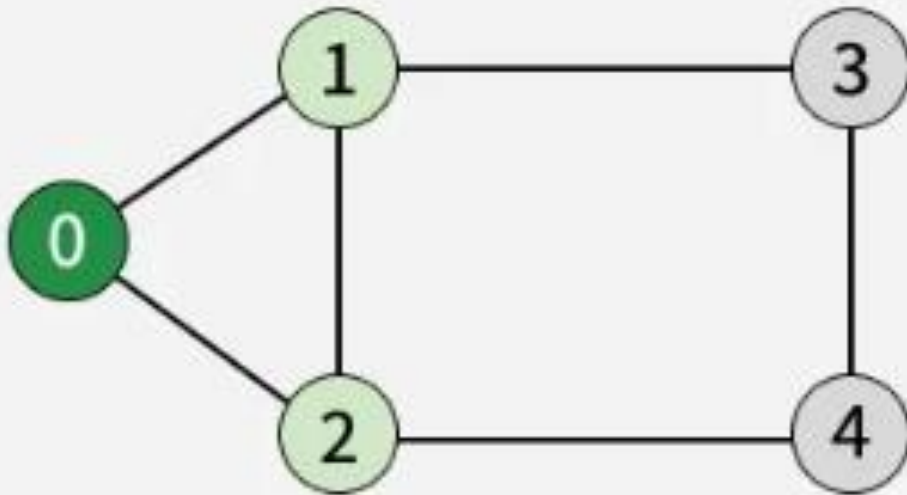
Given the following graph, find its BFS traversal



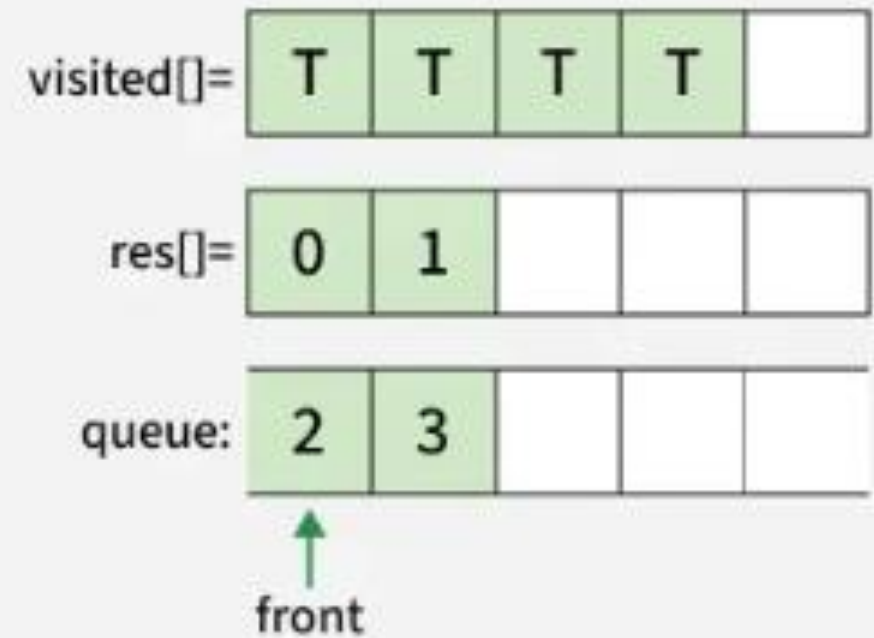
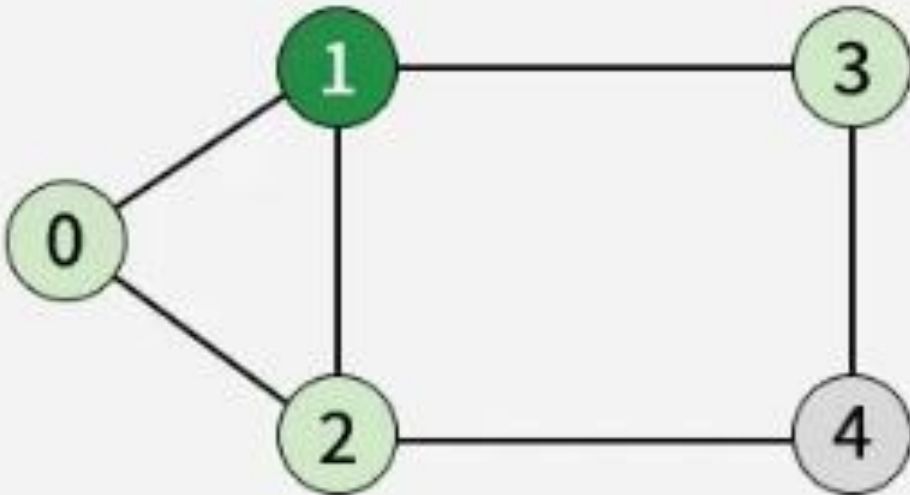
02 | Start with source vertex 0 and push it into the queue



03 | Process node 0, and push its unvisited neighbours into the queue.

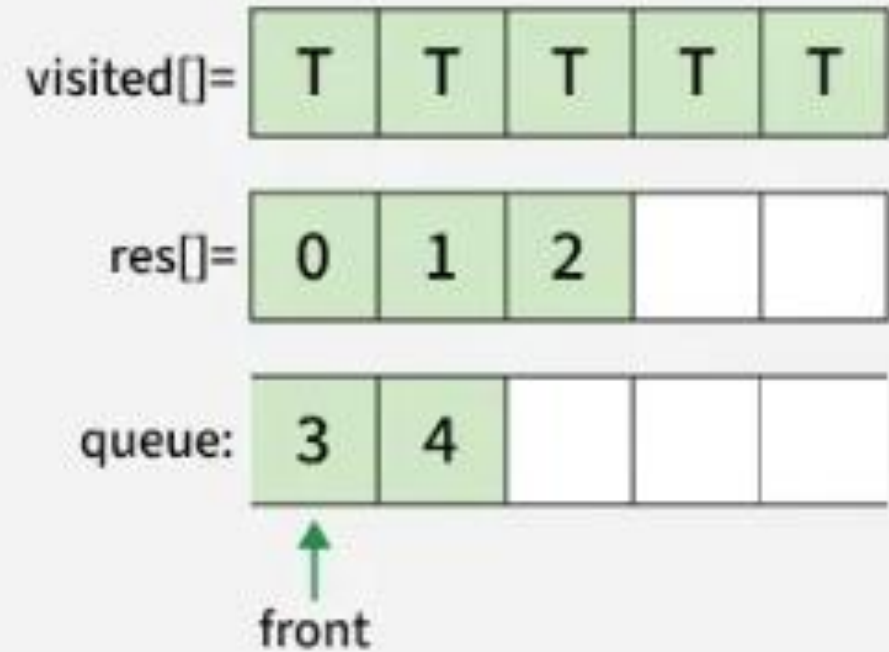
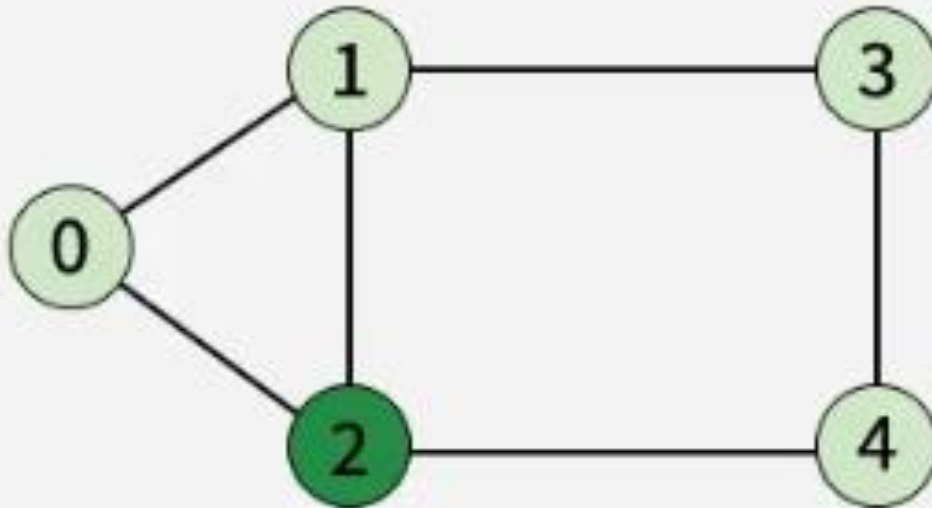


04 | Process node 1, and push its unvisited neighbours into the queue.

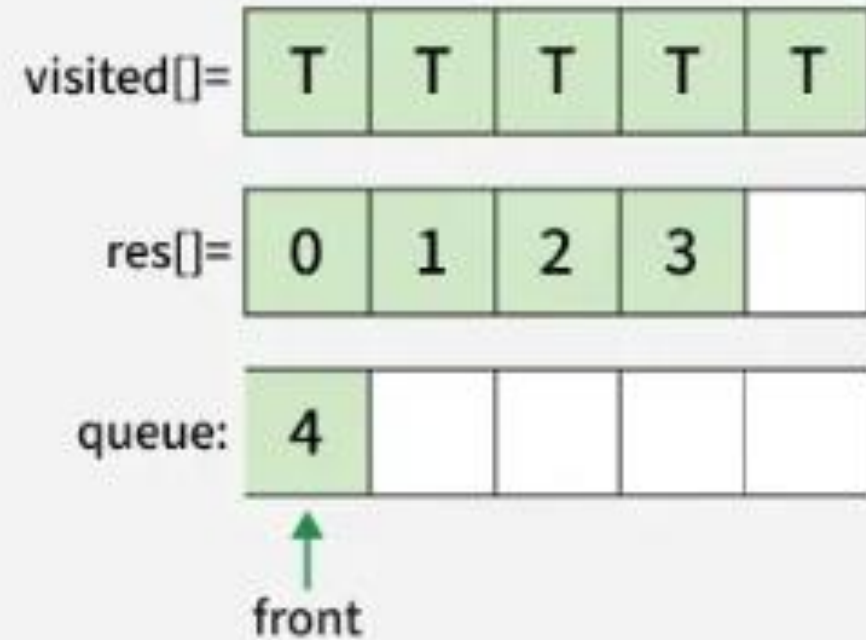
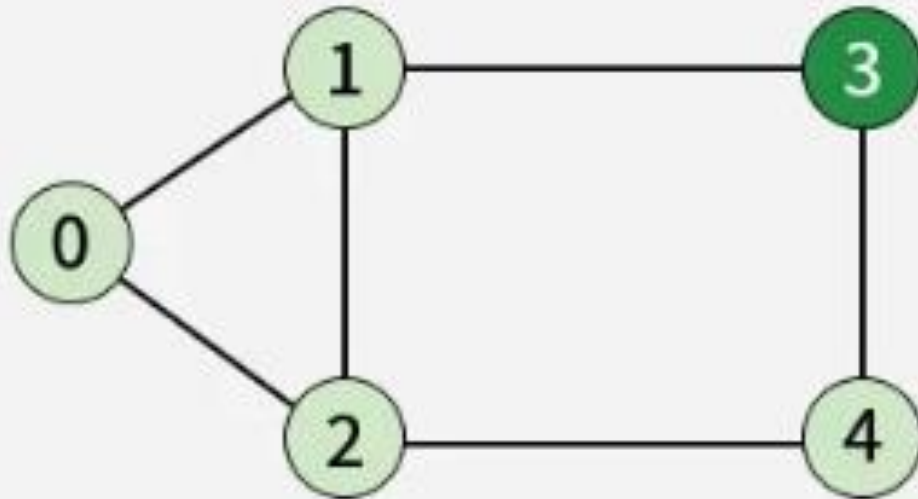


05

Process node 2, and push its unvisited neighbours into the queue.

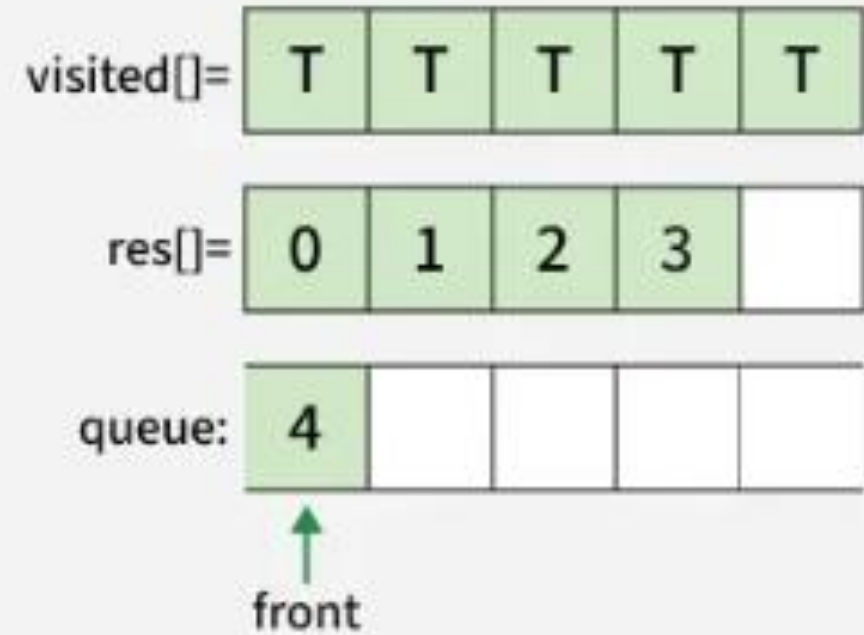
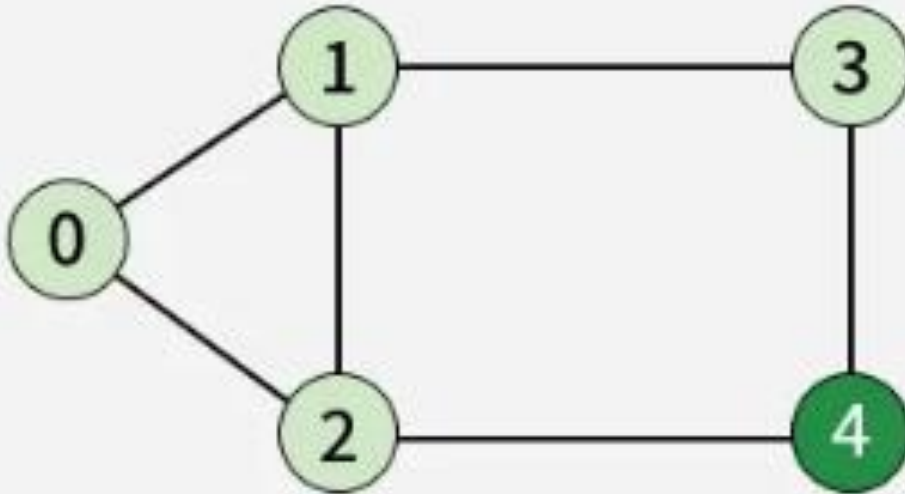


06 | Process node 3, and push its unvisited neighbours into the queue.



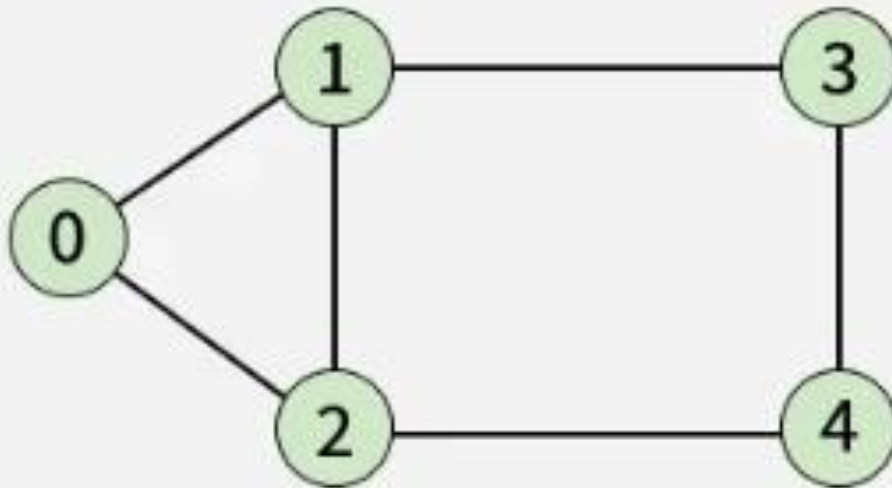
07

Process node 4, and push its unvisited neighbours into the queue.



08

All the neighbours of 4 have been visited, proceed to the next node in the queue. Now that the queue is empty and there are no more nodes to process, we obtain the final BFS traversal.



BFS traversal order is 0, 1, 2, 3, 4.

visited[]=	T	T	T	T	T
res[]=	0	1	2	3	4
queue:					

1 Adjacency Matrix Representation

◆ Definition:

An adjacency matrix is a 2D array (matrix) of size $V \times V$, where:

- If there is an edge between vertex i and j , then

$$A[i][j] = 1$$

- Else

$$A[i][j] = 0$$

For weighted graphs, we store weight instead of 1.

◆ Example Graph:

Vertices: A, B, C, D

Edges: (A–B), (A–C), (B–D), (C–D)

◆ Adjacency Matrix:

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

◆ Space Complexity:

$$O(V^2)$$

◆ Advantages:

- Simple representation
 - Easy to check whether an edge exists
 - Suitable for dense graphs
-

◆ Disadvantages:

- Wastes space for sparse graphs
- Uses more memory

2 Adjacency List Representation

◆ Definition:

An adjacency list stores a list of neighbors for each vertex.

Each vertex has a linked list / array / vector of adjacent vertices.

◆ Example (Same Graph):

CSS

A → B → C

B → A → D

C → A → D

D → B → C

◆ Diagram Form:

Vertex	Adjacent Nodes
A	B, C
B	A, D
C	A, D
D	B, C

◆ Space Complexity:

$$O(V + E)$$

Spanning Tree

A Spanning Tree is a fundamental concept in Graph Theory.

◆ Definition

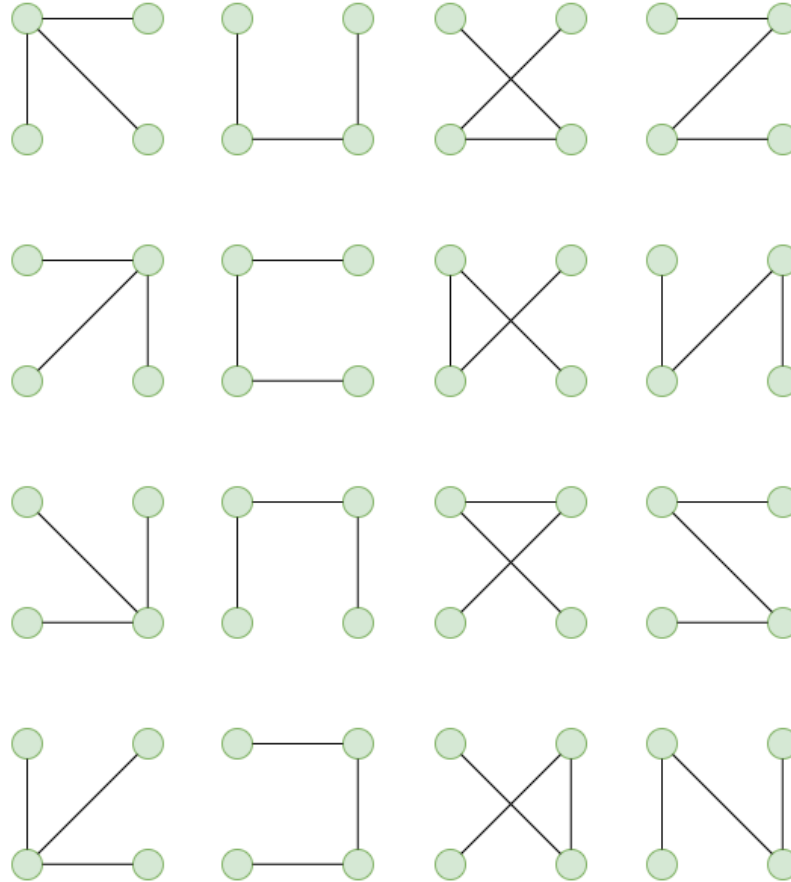
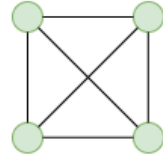
A spanning tree of a graph is a **subgraph** that:

- Includes **all the vertices** of the original graph
- Is **connected**
- Has **no cycles** (i.e., it is a tree)

◆ Key Properties

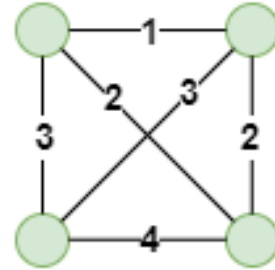
1. If a graph has **n vertices**, a spanning tree will have **exactly $n - 1$ edges**
2. There can be **multiple spanning trees** for a single graph
3. Removing any edge from a spanning tree → it becomes **disconnected**
4. Adding any edge → it creates a **cycle**

Graph =

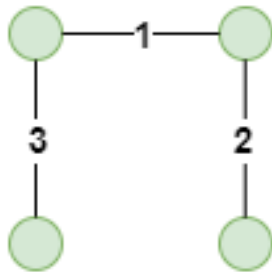


All Possible Spanning Trees of the Graph

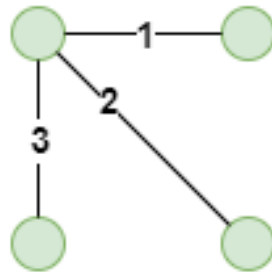
Graph(V,E) =



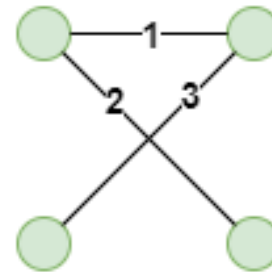
All Possible MST's of the above Graph



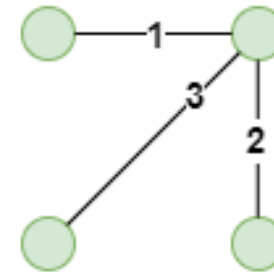
MST Cost =6



MST Cost =6



MST Cost =6



MST Cost =6

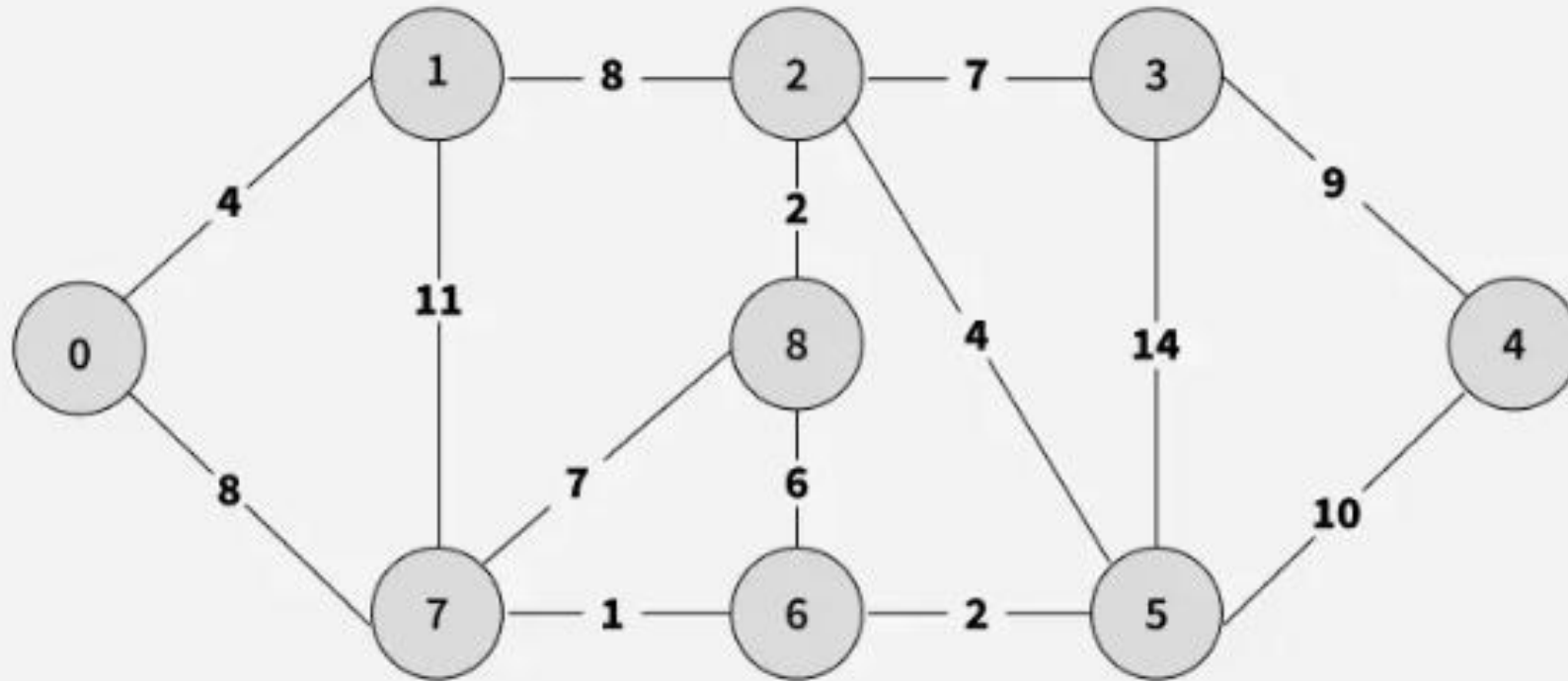
Kruskal's Minimum Spanning Tree (MST) Algorithm

A minimum spanning tree (MST) or minimum weight spanning tree for a weighted, connected, and undirected graph is a spanning tree (no cycles and connects all vertices) that has minimum weight. The weight of a spanning tree is the sum of all edges in the tree.

Below are the steps for finding MST using Kruskal's algorithm:

- Sort all the edges in a non-decreasing order of their weight.
- Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far.
- If the cycle is not formed, include this edge. Else, discard it. It uses the Disjoint Sets to detect cycles.
- Repeat step 2 until there are $(V-1)$ edges in the spanning tree.

The graph contains 9 vertices and 14 edges. So, the minimum spanning tree formed will be having $(9 - 1) = 8$ edges.



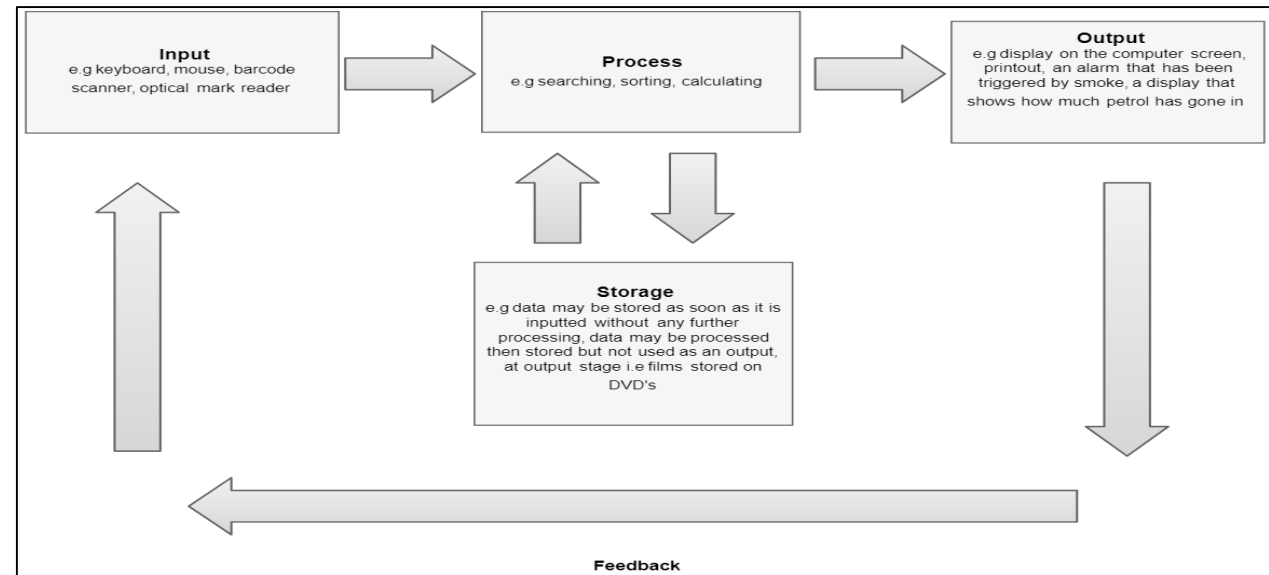


L P U



Understanding Algorithms

- Definition: Step-by-step instructions to solve problems
- Input → Process → Output
- Deterministic vs Non-Deterministic Algorithms
- Importance in problem-solving
- Foundational to computer science
- Examples: Sorting, Searching



An algorithm is a finite sequence of unambiguous instructions used to solve a problem in a systematic manner.

Characteristics of a Good Algorithm

1.Input – Takes input

2.Output – Produces output

3.Definiteness – Steps are clear

4.Finiteness – Ends after finite steps

5.Effectiveness – Each step is simple and executable

Example (Real Life) :

Making Tea Algorithm:

- 1.Boil water
- 2.Add tea leaves
- 3.Add sugar and milk
- 4.Stir and boil for 2-3 minutes
- 5.Strain and serve

Analysis of Algorithms

It is the process of studying:

- **How much time** an algorithm takes to run
- **How much memory** it uses

↳ Helps in choosing the **best algorithm** among multiple solutions.

◆ Types of Analysis

1. Time Complexity 🕒

Measures how execution time increases with input size.

👉 Expressed using Big O Notation

Common Cases:

- **Best Case** – Minimum time taken
- **Average Case** – Typical time taken
- **Worst Case** – Maximum time taken

Examples:

- $O(1)$ → Constant time (fastest)
- $O(n)$ → Linear time
- $O(n^2)$ → Quadratic time (slower)

2. Space Complexity

Measures how much memory an algorithm needs.

Includes:

- Input space
- Auxiliary space (extra memory used)

◆ Example

Linear Search:

- Check each element one by one
- **Time Complexity:**
 - Best Case $\rightarrow O(1)$
 - Worst Case $\rightarrow O(n)$

Running Time Analysis

Running Time Analysis is a part of algorithm analysis that focuses on how long an algorithm takes to execute as the size of input increases.

◆ What is Running Time?

👉 Running time = Number of basic operations performed by an algorithm

Instead of measuring in seconds, we count **steps (operations)** because actual time depends on hardware.

◆ How it is Measured?

Using Big O Notation

It shows how running time grows with input size (n).

◆ Common Running Times

Complexity	Meaning	Example
$O(1)$	Constant time	Accessing array element
$O(n)$	Linear time	Linear search
$O(\log n)$	Logarithmic	Binary search
$O(n^2)$	Quadratic	Nested loops

◆ Steps in Running Time Analysis

1. Identify input size (n)
 2. Count basic operations
 3. Express result using Big O
-

◆ Types of Running Time Analysis

- Best Case → Minimum time
 - Average Case → Expected time
 - Worst Case → Maximum time (most important)
-

◆ Key Point

👉 Running time analysis helps to predict performance without actually executing the program.

◆ Graph of Common Time Complexities

Here's how different complexities grow with input size (n):

$$y = 1, \quad y = \log n, \quad y = n, \quad y = n^2$$

Interpretation:

- $O(1)$ → Flat line (constant, fastest)
- $O(\log n)$ → Slow growth
- $O(n)$ → Straight increasing line
- $O(n^2)$ → Curves upward (slow for large n)

◆ Example 1: Single Loop

```
for i = 1 to n:  
    print(i)
```

👉 Runs **n times**

✓ Running Time = $O(n)$

◆ Example 2: Nested Loop

```
for i = 1 to n:  
    for j = 1 to n:  
        print(i, j)
```

👉 Runs **$n \times n = n^2$ times**

✓ Running Time = $O(n^2)$

◆ Example 3: Logarithmic Case

```
i = 1
while i < n:
    i = i * 2
```

👉 Values: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots$

✓ Running Time = $O(\log n)$

◆ Example 4: Mixed Case

```
for i = 1 to n:
    print(i)

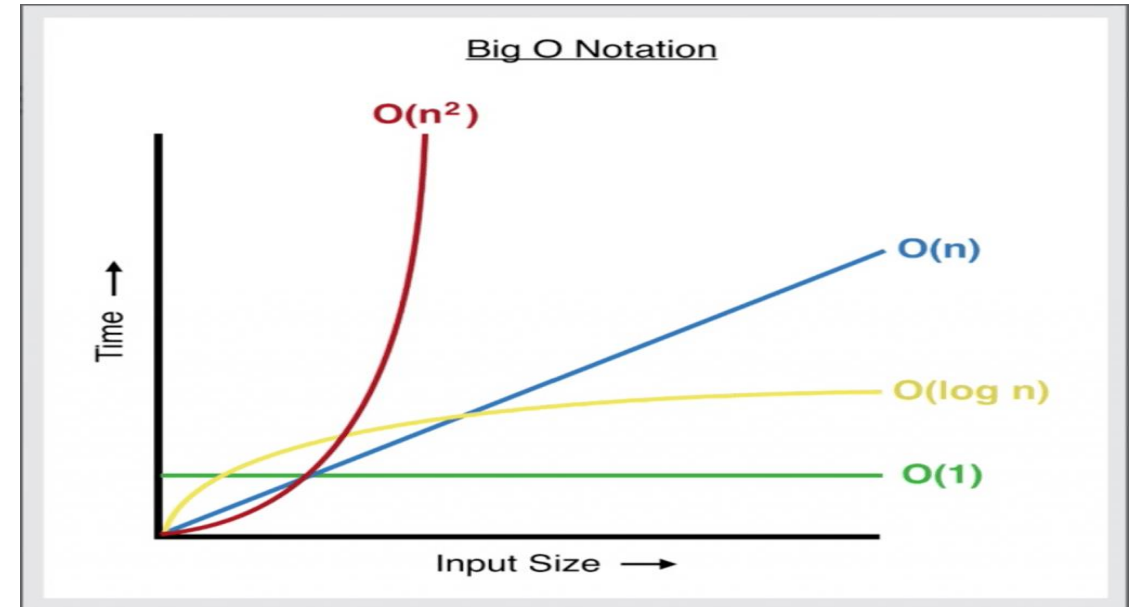
for j = 1 to n:
    print(j)
```

👉 Total operations = $n + n = 2n$

✓ Running Time = $O(n)$ (ignore constants)

Rate of Growth

- **Rate of Growth** is a concept used in algorithm analysis to describe **how quickly the running time (or memory usage) increases as input size (n) increases.**



◆ Measured Using

👉 Big O Notation

◆ Common Growth Rates (from best to worst)

Complexity	Growth Rate	Meaning
$O(1)$	Very slow growth	Constant time (best)
$O(\log n)$	Slow growth	Efficient
$O(n)$	Moderate	Linear
$O(n \log n)$	थोड़ा faster than n^2	Sorting algorithms
$O(n^2)$	Fast growth	Nested loops
$O(2^n)$	Very fast	Exponential (worst)

Asymptotic Notation

Asymptotic Notation is used to describe the performance (time or space) of an algorithm for very large input size (n).

◆ Simple Definition

👉 Asymptotic notation tells us how an algorithm behaves when input becomes very large, ignoring constants and small details.

◆ Why Do We Use It?

- To compare algorithms
- To focus on rate of growth
- To ignore machine-dependent factors (like CPU speed)

Asymptotic Notations

📌 Asymptotic Notations – Simple Explanation

Asymptotic notations are mathematical tools used to analyze the time and space complexity of algorithms, especially when the input size (n) becomes very large.

They help us understand how fast an algorithm grows and how efficient it is.

◆ Why Asymptotic Notations?

- To compare efficiency of algorithms
- To predict performance for large input sizes
- To analyze best, average, and worst-case scenarios

Types of Asymptotic Notations

1 Big-O Notation – $O(f(n))$

 Worst-case time complexity

It gives the upper bound of an algorithm.

Example:

Linear Search $\rightarrow O(n)$

Binary Search $\rightarrow O(\log n)$

Bubble Sort $\rightarrow O(n^2)$

Definition:

Big-O represents the maximum time an algorithm may take.

Big-O Notation

- $O(1)$: Constant time (e.g., array access)
- $O(n)$: Linear time (e.g., traversal)
- $O(\log n)$: Logarithmic time (e.g., binary search)
- $O(n^2)$: Quadratic time (e.g., nested loops)
- $O(2^n)$: Exponential time (complex problems)

2 Omega Notation – $\Omega(f(n))$

→ Best-case time complexity

It gives the lower bound of an algorithm.

Example:

Linear Search (best case) → $\Omega(1)$

Bubble Sort (best case) → $\Omega(n)$

Definition:

Omega represents the minimum time required.

3 Theta Notation – $\Theta(f(n))$

→ Average / Exact bound

It gives tight bound (both upper and lower).

Example:

Merge Sort $\rightarrow \Theta(n \log n)$

Definition:

Theta represents the exact growth rate.

Common Time Complexities

Notation	Name	Example
$O(1)$	Constant	Stack push
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Linear search
$O(n \log n)$	Linearithmic	Merge sort
$O(n^2)$	Quadratic	Bubble sort
$O(2^n)$	Exponential	Recursive Fibonacci

Master Theorem — Simple & Exam-Ready Explanation

The Master Theorem is a formula used to find the time complexity of recursive algorithms, especially those written using divide and conquer strategy.

◆ General Form of Recurrence Relation

Most divide-and-conquer algorithms follow this form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Where:

- $T(n)$ = Time complexity
- a = Number of subproblems
- n/b = Size of each subproblem
- $f(n)$ = Cost of dividing + combining

Master Theorem Cases

◆ Case 1:

If

$$f(n) = O(n^{\log_b a - \varepsilon})$$

Then:

$$T(n) = \Theta(n^{\log_b a})$$

👉 Divide cost dominates

◆ **Case 2:**

If

$$f(n) = \Theta(n^{\log_b a})$$

Then:

$$T(n) = \Theta(n^{\log_b a} \log n)$$

👉 **Balanced case**

◆ Case 3:

If

$$f(n) = \Omega(n^{\log_b a + \varepsilon})$$

and regularity condition holds

Then:

$$T(n) = \Theta(f(n))$$

👉 Combine cost dominates

How to Apply Master Theorem (Steps)

1. Identify $a, b, f(n)$
 2. Compute $n^{\{\log_b a\}}$
 3. Compare $f(n)$ with $n^{\{\log_b a\}}$
 4. Apply correct case
-

Examples

Example 1:

$$T(n) = 2T(n/2) + n$$

- $a = 2, b = 2, f(n) = n$
- $n^{\{\log_2 2\}} = n^1 = n$

👉 Case 2 $\rightarrow \Theta(n \log n)$

Example 2:

$$T(n) = 4T(n/2) + n$$

- $a = 4, b = 2$
- $n^{\log_2 4} = n^2$

👉 $f(n) = n < n^2 \rightarrow \text{Case 1} \rightarrow \Theta(n^2)$

Example 3:

$$T(n) = 2T(n/2) + n^2$$

- $a = 2, b = 2$
- $n^{\log_2 2} = n$

👉 $f(n) = n^2 > n \rightarrow \text{Case 3} \rightarrow \Theta(n^2)$

Common Algorithms Using Master Theorem

Algorithm	Recurrence	Time
Merge Sort	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$
Binary Search	$T(n) = T(n/2) + 1$	$\Theta(\log n)$
Quick Sort (avg)	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$



Searching Algorithms

- Linear Search: Sequential search ($O(n)$)
- Binary Search: Divide and conquer ($O(\log n)$)
- Works on sorted data
- Applications: Searching records
- Comparison: Linear vs Binary Search

Sorting Algorithms

- Bubble Sort: Simple, inefficient ($O(n^2)$)
- Insertion Sort: Builds sorted list ($O(n^2)$)
- Selection Sort: Finds minimum iteratively
- Merge Sort: Divide and conquer ($O(n \log n)$)
- Quick Sort: Efficient for large data ($O(n \log n)$)

Dynamic Programming

Dynamic Programming (DP) is a powerful technique used to solve complex problems by breaking them into smaller subproblems, solving each once, and storing the results to avoid repeated work.

◆ Simple Definition

👉 Dynamic Programming = Divide + Store + Reuse

◆ Key Idea

Instead of solving the same problem again and again:

- Solve it once
- Store the result
- Reuse it when needed

◆ Two Approaches

1. Top-Down (Memoization)

- Uses recursion
- Stores results in memory

2. Bottom-Up (Tabulation)

- Uses iteration
- Builds solution step-by-step

◆ Classic Example: Fibonacci Series

Without DP (Inefficient)

$$F(n) = F(n-1) + F(n-2)$$

👉 Repeats calculations → Slow

Saving Money 💰

👉 Suppose you track your savings every month:

- Month 1 → ₹1000
- Month 2 → ₹2000
- Month 3 → ₹3000

Instead of recalculating total savings every time:

✓ You store previous totals and add new values

👉 Reuse past results = DP concept

Exam Preparation

👉 You prepare chapter-wise:

- First learn basics
- Then build advanced concepts on top

✓ You reuse previous knowledge instead of starting from scratch

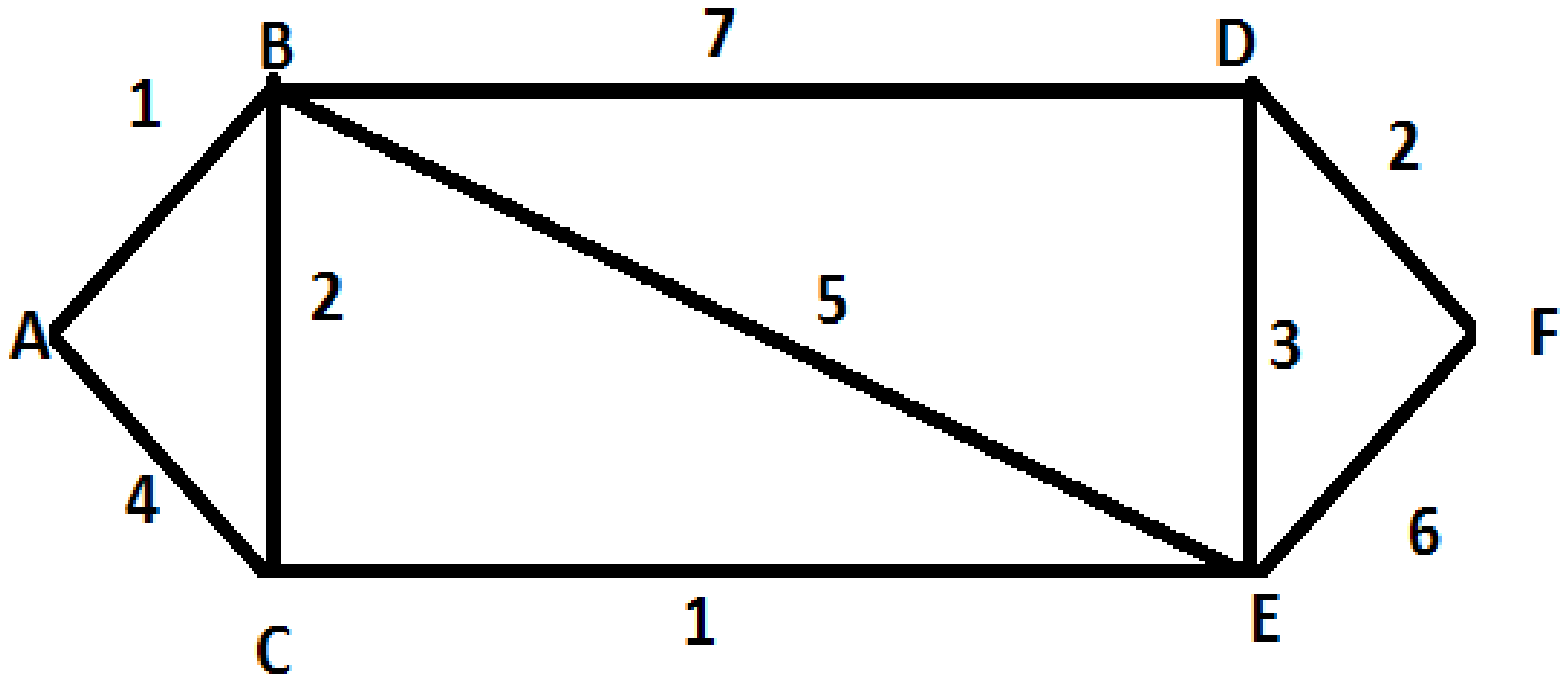
👉 This is similar to building solutions step-by-step (DP)



Shortest Path Algorithms

- Dijkstra's Algorithm: Weighted graphs
- Bellman-Ford Algorithm
- Applications: GPS, Maps
- Examples: Travel optimization
- Visual flow of shortest path

Using Dijkstra Algorithm find the cost of path from A to F



Final Shortest Path

A → B → C → E → D → F

Total Distance:

$$1+2+1+3+2=9$$

Shortest Distance = 9



Greedy Algorithms

- Choose best now, don't worry about future
- Solve problems step by step
- Always chooses the locally optimal solution
- Example: Coin change problem

◆ 1. ATM Cash Withdrawal 💰

👉 You withdraw ₹270 from an ATM:

- Take ₹200 first
- Then ₹50
- Then ₹20

✓ Always choosing the **largest possible note first**

👉 This is a **greedy choice**

◆ 2. Filling a Bag with Items 🎒 (Fractional Knapsack)

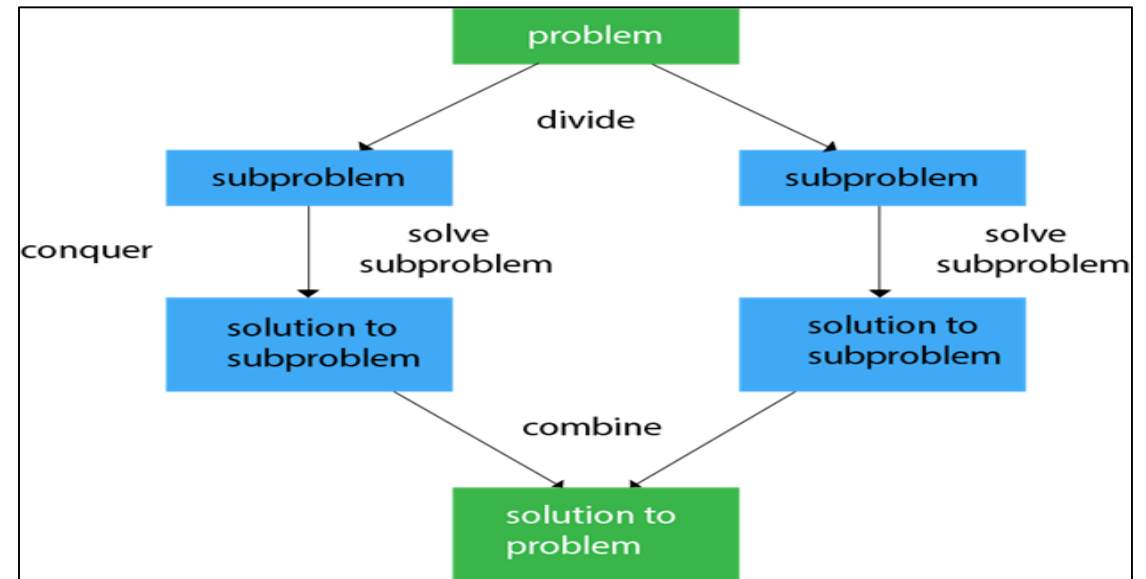
👉 You want maximum value in limited weight:

- Pick item with **highest value per kg first**
- Then next best

✓ Step-by-step best choice → Greedy

Divide and Conquer Algorithms

- Divides problem into smaller subproblems
- Solves recursively and combines solutions
- Examples: Merge Sort, Quick Sort
- Used in solving complex problems
- Reduces time complexity
- Diagram: Divide and Conquer process



Sort In-Place (In-Place Sorting)

◆ Definition:

In-place sorting means sorting the elements **within the original data structure itself**, without using significant extra memory.

 Only **constant extra space ($O(1)$)** is used.

◆ Key Idea:

- Sorting is done **inside the same array**
- No additional array or large memory is required

◆ Examples of In-Place Sorting Algorithms:

Algorithm	In-Place?
Selection Sort	✓ Yes
Insertion Sort	✓ Yes
Bubble Sort	✓ Yes
Quick Sort	✓ Yes (<i>uses small recursion stack</i>)
Heap Sort	✓ Yes
Merge Sort	✗ No (<i>needs extra array</i>)

◆ Example:

Original Array:

```
[5, 3, 1, 4, 2]
```

After In-Place Sorting:

```
[1, 2, 3, 4, 5]
```

✓ Sorting is done in the same array, no new array created.

Sorting Algorithms & Analysis

1. Quick Sort

👉 Quick Sort selects a **pivot element**, then:

- Places smaller elements on the left
 - Larger elements on the right
 - Repeats the process for subarrays
-

◆ Working Steps

1. Choose a **pivot element**
2. Partition the array:
 - Elements $<$ pivot $\rightarrow$ left
 - Elements $>$ pivot $\rightarrow$ right
3. Recursively apply same steps

◆ Example

Sort: [8, 3, 1, 7, 0, 10, 2]

- Choose pivot = 7
- Left: [3, 1, 0, 2]
- Right: [8, 10]

Now repeat for left and right parts until sorted:

✓ Final Output: [0, 1, 2, 3, 7, 8, 10]

◆ Summary Table

Case	Time Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$

2. Merge Sort

◆ Concept

Merge Sort also uses **divide and conquer**:

1. Divide array into halves.
2. Recursively sort both halves.
3. Merge the sorted halves.

◆ Time Complexity

Case	Complexity
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$

◆ Space Complexity

$$O(n)$$

4. Insertion Sort

◆ Concept

Builds sorted array one element at a time.

◆ Time Complexity

Case	Complexity
Best	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$

◆ Working Idea

- Take one element
- Compare it with previous elements
- Insert it in the correct place

◆ Example

Sort: [5, 2, 4, 6, 1, 3]

Step-by-step:

- Pass 1: [2, 5, 4, 6, 1, 3]
- Pass 2: [2, 4, 5, 6, 1, 3]
- Pass 3: [2, 4, 5, 6, 1, 3]
- Pass 4: [1, 2, 4, 5, 6, 3]
- Pass 5: [1, 2, 3, 4, 5, 6] ✓

5. Selection Sort

◆ Concept

Repeatedly selects minimum element and places it at correct position.

◆ Time Complexity

Case	Complexity
Best	$O(n^2)$
Average	$O(n^2)$
Worst	$O(n^2)$

◆ Example

Sort: [64, 25, 12, 22, 11]

Step-by-step:

- Pass 1: smallest = 11 → [11, 25, 12, 22, 64]
- Pass 2: smallest = 12 → [11, 12, 25, 22, 64]
- Pass 3: smallest = 22 → [11, 12, 22, 25, 64]
- Pass 4: smallest = 25 → [11, 12, 22, 25, 64] ✓

Comparison of Sorting Techniques

Algorithm	Best	Avg	Worst	Space
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$

1. Linear Search (Sequential Search)

◆ Definition

Linear search is a **simple searching algorithm** that checks **each element one by one** until the required element is found or the list ends.

◆ Algorithm Steps

1. Start from the first element.
2. Compare each element with the key.
3. If match → element found.
4. If end reached → element not found.

◆ Time Complexity

Case	Comparisons	Complexity
Best	1	O(1)
Average	$(n + 1) / 2$	O(n)
Worst	n	O(n)

◆ Average Number of Comparisons (Successful Search)

$$\frac{1 + 2 + 3 + \dots + n}{n} = \frac{n + 1}{2}$$

2. Binary Search

◆ Definition

Binary search is a **fast searching algorithm** that works only on **sorted arrays** by repeatedly dividing the search interval into halves.

◆ Precondition

→ Data must be **sorted**

◆ Algorithm Steps

1. Find middle element.
2. Compare key with middle.
3. If equal $\rightarrow$ found.
4. If smaller $\rightarrow$ search left half.
5. If greater $\rightarrow$ search right half.
6. Repeat until found or interval empty.

◆ Recurrence Relation

$$T(n) = T(n/2) + O(1)$$

◆ Time Complexity

Case	Comparisons	Complexity
Best	1	$O(1)$
Average	$\log_2(n)$	$O(\log n)$
Worst	$\log_2(n)$	$O(\log n)$

Average Number of Comparisons for Binary Search

Formula:

$$\text{Average Comparisons} = \log_2(n)$$

More precisely:

$$= \log_2(n + 1)$$

Where n = number of elements

Example:

If $n = 16$

$$\log_2(16) = 4$$

So, average comparisons = 4

What is an AVL Tree?

An **AVL Tree** is a self-balancing Binary Search Tree (BST).

AVL = Adelson-Velsky and Landis (the scientists who invented it)

◆ Definition:

An **AVL tree** is a **binary search tree** in which the **height difference (balance factor)** between the **left and right subtrees of every node** is at most 1.

$$\text{Balance Factor (BF)} = \text{Height}(\text{left subtree}) - \text{Height}(\text{right subtree})$$

Allowed values of BF:

$$-1, 0, +1$$

◆ What is a Hash Table?

- 👉 A hash table stores data in **key → value** pairs
- 👉 It uses a **hash function** to decide where to store data

◆ Real-Life Example 📖 (Phone Directory)

Name (Key)	Phone Number (Value)
Aman	98765
Ravi	91234
Neha	99887

- 👉 You search by **name (key)** and get **number (value)**

Hash Function & Clustering

◆ What is a Hash Function?

A hash function $h(k)$ maps a key k into an index of a hash table.

$$h(k) \rightarrow \text{Index location in hash table}$$

◆ What is Clustering?

Clustering refers to the **grouping of keys in consecutive slots** of a hash table, leading to **performance degradation**.

◆ Which hash function creates clustering?

$$h(k) = k$$

- If keys are sequential $\rightarrow$ slots become sequential.
- This creates **primary clustering**.
- Hence, it is a **poor hash function**.

✓ Conclusion:

$h(k) = k \rightarrow$ causes clustering

2. Load Factor of Hash Table

◆ Definition:

Load factor (α) indicates how full the hash table is.

$$\alpha = \frac{\text{Number of elements}}{\text{Table size}}$$

◆ Example:

Hash table slots = 25

Elements = 2000

$$\alpha = \frac{2000}{25} = 80$$

✓ Load Factor = 80

3. Probability of Collision (Birthday Paradox Concept)

◆ Formula (Approximation):

$$n \approx \sqrt{m}$$

Where:

m = size of hash table

n = number of inserted keys

◆ Example:

Table size = 50

$$n \approx \sqrt{50} \approx 7$$

Closest option = 10

✓ Probability of collision ≈ 0.5 when ~10 keys inserted

◆ Simple Numeric Example

Let's store values in a table of size 10

👉 Hash function:

$$h(\text{key}) = \text{key} \bmod 10$$

Insert Elements: 25, 36, 48

- $h(25) = 25 \% 10 = 5 \rightarrow \text{index } 5$
- $h(36) = 36 \% 10 = 6 \rightarrow \text{index } 6$
- $h(48) = 48 \% 10 = 8 \rightarrow \text{index } 8$

✓ Stored positions:

Index	Value
5	25
6	36
8	48

◆ Collision Example ⚠

Insert 35:

- $h(35) = 35 \% 10 = 5$
👉 But index 5 already has 25

✓ This is called collision

◆ Collision Handling Methods

1. Chaining → Store multiple values in same index (list)
2. Linear Probing → Find next empty space

◆ What is Linear Probing?

👉 When a collision occurs, we check the next empty slot sequentially.

👉 Formula:

- $h(\text{key}) = \text{key} \% \text{table size}$
- If occupied $\rightarrow$ check $(\text{index} + 1)$, $(\text{index} + 2)$, ...

◆ Example

Given:

- Table size = 7
- Hash function:

👉 $h(\text{key}) = \text{key} \bmod 7$

Insert elements: 10, 17, 24, 31

◆ Final Hash Table

Index	Value
0	—
1	—
2	—
3	10
4	17
5	24
6	31

Hash Table Insertion Using Linear Probing

Given:

- Hash table size = 10
- String = KRPY SNJM
- Hash function:

$$h(x) = (ord(x) - ord("A") + 1) \mod 10$$

Step-by-step Hash Value Calculation:

Character	Alphabet Position	$h(x)$
K	11	1
R	18	8
P	16	6
Y	25	5
S	19	9
N	14	4
J	10	0
M	13	3

Final Hash Table:

Index	Value
0	J
1	K
3	M
4	N
5	Y
6	P
8	R
9	S

What is Greedy Programming?

A **Greedy Algorithm** is an algorithmic approach that **selects the best possible choice at each step** with the hope of achieving a **globally optimal solution**.

Key Idea:

Make locally optimal choice → Hope to reach global optimum

Greedy Algorithm Works Best

Greedy algorithms work efficiently when:

- The problem has **Greedy Choice Property**
- The problem shows **Optimal Substructure**

Problems Efficiently Solved Using Greedy Programming

Problem	Greedy Solution
Fractional Knapsack	✓ Optimal
Huffman Coding	✓ Optimal
Prim's Algorithm (MST)	✓
Kruskal's Algorithm (MST)	✓
Dijkstra's Algorithm (Shortest Path)	✓
Job Sequencing with Deadlines	✓

Problems NOT Solved Using Greedy Programming

Problem	Reason
Traveling Salesman Problem (TSP)	NP-Hard, greedy fails
0/1 Knapsack	Needs DP
All-pairs shortest path	Needs DP

Greedy Algorithm for Huffman Coding

◆ Purpose:

Used for data compression.

◆ Method:

- Use min-heap / priority queue
- Combine two least frequent nodes repeatedly

◆ Time Complexity:

$$O(n \log n)$$

Characteristics of Greedy Algorithms

- Makes **locally optimal decisions**
- Simple and easy to implement
- Faster than dynamic programming and backtracking
- Does **not always guarantee optimal solution**
- Works only for **specific classes of problems**

Time Complexity Summary

Algorithm	Time Complexity
Huffman Coding	$O(n \log n)$
Fractional Knapsack	$O(n \log n)$
Prim's Algorithm	$O(E \log V)$
Dijkstra's Algorithm	$O(E \log V)$

Huffman Tree Construction

Symbol	Frequency
A	7
B	9
C	11
D	14
E	18
F	21
G	27
H	29
I	35
J	40

◆ Step-by-Step Huffman Merging Process

At each step, pick the two smallest frequencies, merge them, and reinsert.

Step 1

$A(7) + B(9) \rightarrow 16$

Queue $\rightarrow 11, 14, 16, 18, 21, 27, 29, 35, 40$

Step 2

$C(11) + D(14) \rightarrow 25$

Queue $\rightarrow 16, 18, 21, 25, 27, 29, 35, 40$

Step 3

$$16 + 18 \rightarrow 34$$

Queue $\rightarrow$ 21, 25, 27, 29, **34**, 35, 40

Step 4

$$21 + 25 \rightarrow 46$$

Queue $\rightarrow$ 27, 29, **34**, 35, 40, **46**

Step 5

$$27 + 29 \rightarrow 56$$

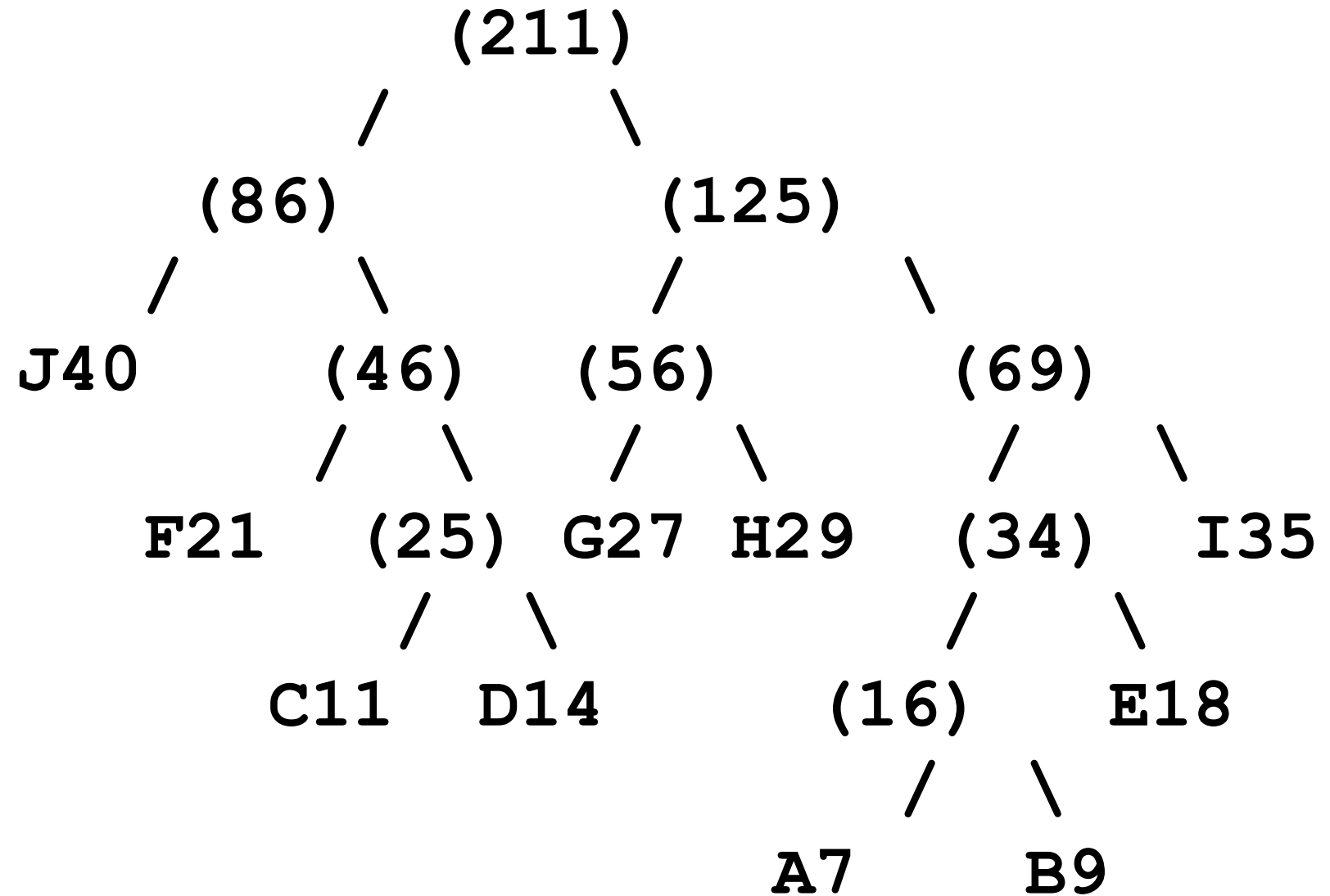
Queue $\rightarrow$ **34**, 35, 40, **46**, **56**

**Step 6** $34 + 35 \rightarrow 69$ Queue $\rightarrow 40, 46, 56, 69$

Step 7 $40 + 46 \rightarrow 86$ Queue $\rightarrow 56, 69, 86$

Step 8 $56 + 69 \rightarrow 125$ Queue $\rightarrow 86, 125$

Step 9 (Final Merge) $86 + 125 \rightarrow 211$ (ROOT)



Problem Definition

The 0/1 Knapsack Problem can be stated as:

- You have n items, each with:
 - a weight $w[i]$
 - a value $v[i]$
- You have a knapsack (bag) with capacity W .
- You want to maximize the total value of items you can put in the knapsack, without exceeding the capacity.
- Constraint: Each item can be either included or excluded (0 or 1). You cannot take fractions of items.

Example

Item	Weight	Value
1	2	3
2	3	4
3	4	5
4	5	8

Knapsack capacity $w = 5$

- We can include item 1 and 2? weight = $2+3 = 5$ ✓ value = $3+4=7$
- Or include item 4 alone? weight=5 ✓ value=8 ✓ (Better!)

Maximum value = 8



Real-World Challenges

- Balancing time vs space efficiency
- Handling edge cases in inputs
- Solving scalability issues for Big Data
- Debugging algorithm implementation
- Efficiency for real-world applications
- Example: AI and Data Analysis



Summary of Key Concepts

- Algorithms: Step-by-step problem-solving
- Data Structures: Organize and store data
- Big-O: Measure algorithm efficiency
- Trees, BSTs, Stacks, Queues: Core structures
- Searching and Sorting explained
- Applications in real life

Web Links For Reference (Complete Video Course)

- https://youtube.com/playlist?list=PLZ2ps__7DhBYc4jkUk_yQAjYEVFzVzhdU&si=2cdYZiJa6NVwbJ6s
- <https://youtube.com/playlist?list=PLIwC9bZ0rmjSkm1VRJROX4vP2YMIf4Ebh&si=kH6pymcwWOLcGtqP>
- https://youtube.com/playlist?list=PLBlnK6fEyqRi_CUQ-FXxgzKQ1dwr_ZJWZ&si=vRmw_BpdGuBIhDkb

Web Links For Reference (Study Material)

- <https://www.appsierra.com/blog/database-normalization-tutorial>
- <https://www.exasol.com/blog/what-is-a-relational-database/>
- <https://www.geeksforgeeks.org/dbms/>
- <https://db-book.com/slides-dir/>
- <https://www.w3schools.com/sql/>
- <https://www.javatpoint.com/dbms-tutorial>
- <https://byjus.com/govt-exams/database-management-system-dbms/>

Next -> Advanced Algorithms and Problem-Solving Techniques

- Sorting and Searching Algorithms
- Classification and Comparison of Sorting Algorithms
- Linear and Non-Linear Searching Techniques
- Hashing and Hash Table Implementation
- Graph Algorithms and Design Strategies
- Graph Representation and Traversals
- Shortest Path Algorithms
- Introduction to Greedy
- Divide and Conquer
- Dynamic Programming Strategies